Which Pixels Matter More? Entropy-Uniform Rendering for Optical Long-Context Compression

Anonymous CVPR submission

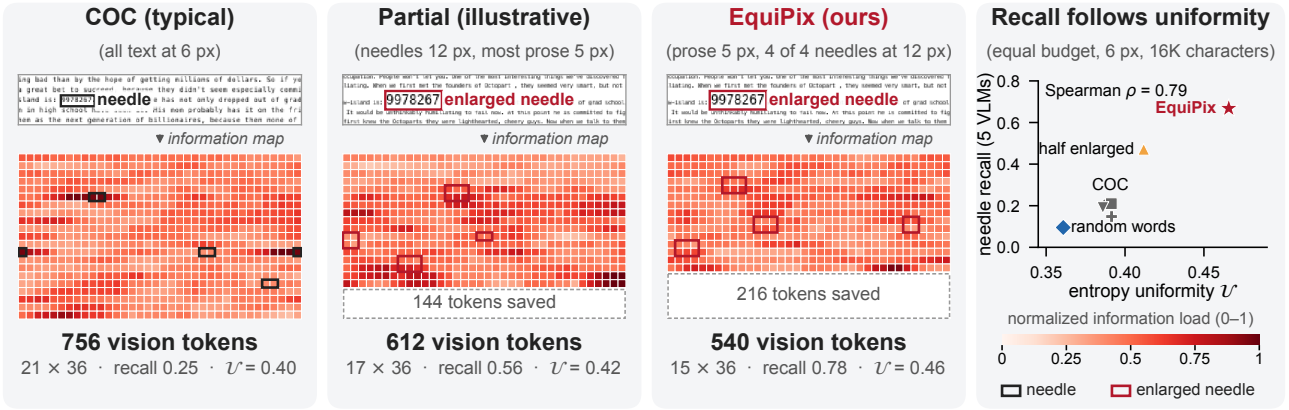


Figure 1. **Information is spread unevenly over a rendered page, and recall follows how evenly it is spread.** One 16K-character RULER document with four seven-digit needles, rendered within the vision-token budget of typical context optical compression (COC). *Top:* crops of the rendered images; black boxes mark the needles and red boxes the enlarged ones. *Bottom:* information maps drawn to scale, one cell per vision token, colored by normalized load (Sec. 3.3); the color is computed from the text and is not part of the image. Under COC the needles are hot spots. The middle rendering is an illustration that enlarges the needles but keeps the first fifth of the prose at 6 px; it was not evaluated. EquiPix shrinks all prose, uses the fewest tokens, and has the flattest map. Recall: five VLMs, 50 documents. *Right:* recall against entropy uniformity $\mathcal{U}$ for every rendering at the 6 px budget (both budgets: Sec. 5.1).

Abstract

Context optical compression (COC) renders long text into images so that a vision-language model (VLM) can read it with several times fewer tokens. Typical COC gives every character the same pixels. We argue that this misallocates the budget, because information is spread unevenly over a rendered page. A reader can recover predictable prose from its language prior, but characters with high residual entropy, such as those of identifiers and hashes, must come from the pixels. We estimate this residual entropy from the text alone, which yields an information map of any rendered page, and we summarize the map with an entropy-uniformity score $\mathcal{U}$. Across 13 rendering–budget combinations of the same documents at an equal vision-token budget, $\mathcal{U}$ tracks needle recall over five VLMs (Spearman $\rho = 0.84$). Font search and downsampling leave $\mathcal{U}$ almost unchanged and barely help, and enlarging random words lowers both. Flattening the map is then the natural remedy. Our method, EquiPix, enlarges high-entropy strings and shrinks prose within the budget of typical COC, without training or model access. It raises RULER needle recall by 0.55 and 0.43 at the two densest budgets, and by 0.48 at 32K–128K characters with up to 27% fewer tokens per task. It also doubles the recall of flagged identifiers in LongBench documents and leaves QA accuracy unchanged. The gain shrinks at milder budgets, and every image-based method still trails plain text on QA.

1. Introduction

Long contexts are expensive to feed to large models. Context optical compression (COC) cuts this cost by rendering text into images and letting a vision-language model (VLM) read the pixels [7, 34, 41]. A page of small glyphs costs a fixed number of vision tokens however many characters it holds. In our measurements a 128K-character essay costs 27.1K prompt tokens as text and 4.7K as a typical COC image (mean of the four readers that report usage for both).

COC needs no training and works with any VLM that accepts images.

Typical COC renders the whole document with one font and one size, chosen at best by a global search [7]. Every character then receives the same pixels. The reader, however, does not need the same evidence for every character. A VLM that reads rendered text combines what the glyph shows with what its language model expects. In prose the expectation fills most gaps, so blurred letters are still read correctly. In an identifier it cannot: the format of a UUID is predictable, but each hexadecimal character carries 4 bits that only the pixels can supply. Our readers show this split. Under typical COC they write the hyphens of a UUID correctly 99% of the time and its hexadecimal characters only 51% of the time, and most wrong answers keep the format and invent the content (Sec. 5.3). At an equal budget, typical COC recovers only 8–22% of RULER needles, averaged over five readers.

We ask which pixels matter more, and answer in four steps (Fig. 1). The first is a theory of where information sits after rendering. What the pixels must supply for a character is roughly its residual entropy given the context, and we estimate it from the text before anything is drawn (Sec. 3.2). The second is a measure. Residual entropy divided by glyph area gives the information load of a region in bits per vision token. Entropy uniformity $\mathcal{U}$ compares the mean load of a page with the load of its hottest regions (Sec. 3.3). $\mathcal{U}$ is 1 when every region carries the same information per pixel and small when a few regions carry far more than the rest. The third is a visualization: the load can be drawn as a heat map of any text under any rendering, which shows the informative and error-prone spans before any model is called. The fourth is the method. If uneven load causes the errors, the remedy is to flatten the map, and EquiPix does so by enlarging high-entropy strings and shrinking prose within the vision-token budget of typical COC (Sec. 3.5). It is training-free, query-independent, and needs only black-box access.

A measure is useful only if it predicts reading. Across 13 arm–budget combinations of the same documents, $\mathcal{U}$ and five-reader recall have Spearman $\rho = 0.84$. Two results do not follow from how EquiPix is built. Font search and render-then-downsample rescale every region about equally; they leave $\mathcal{U}$ almost unchanged and barely improve recall. When a random half of the needles is enlarged, the enlarged ones are recalled 74–83% of the time and the others 11–12% (Sec. 5.1).

Our contributions are:

- an information view of COC, which traces verbatim failures to characters whose residual entropy the language prior cannot supply and estimates that entropy from the text alone;
- entropy uniformity and information maps, which track recall across renderings at an equal budget and explain why rescaling the whole page (font search, downsampling) does not help;
- EquiPix, a budget-constrained glyph allocation that flattens the map. Over five readers it raises needle recall by 0.43–0.55 at 16K characters and by 0.48 at 32K–128K with up to 27% fewer tokens, doubles the recall of flagged natural identifiers, and leaves LongBench QA unchanged. The gain shrinks at milder budgets, and at 10 px font search does as well.

Verbatim anchors, which copy flagged strings into the prompt next to the image, leave the map unchanged; we treat them as optional post-processing (Sec. 5.7).

2. Related Work

Reading text from pixels has a long history, from pixel-based language models [11, 34] to unified image–text encoders [37] and screenshot pretraining [20]. Context optical compression (COC) renders long contexts to compress them: DeepSeek-OCR decodes text from few vision tokens [41, 42], Glyph searches for a good global rendering configuration [7], and other work renders whole inputs, distant context, or interleaved text [25, 39, 45]. Evaluations show where this fails: compressed reading loses retrieval ability [52], the text–pixel gap depends on font and resolution [36], optical compression can underperform mean pooling [19], and accuracy collapses when linguistic priors are removed [12, 26]. This last finding is the starting point of our account: what the prior cannot supply must come from the pixels, and our entropy maps make this quantity visible on the page. Several methods vary resolution within the input, but they train the reader and decide per query over several turns: FocusVTC requests high-resolution crops while it reasons [53], SEER retrieves the original text of relevant pages [46], and LensVLM expands selected pages to full resolution [44]. DocOwl2 and TextMonkey learn high-resolution compression inside the model [16, 28]. Among rendering changes, AGAR is the most similar: it re-renders a page and enlarges the word spans that receive the most middle-to-late-layer attention [48]. It is training-free but needs the model's attention and a first pass for each query. EquiPix is decided from the text before any model call, works with black-box APIs, and does not depend on the question.

Training-free pruning and merging reduce the tokens of an encoded image [4, 5, 10, 29, 35, 47, 49, 50], including for text-rich and OCR inputs [22, 38]. They act on visual features inside the model, where it is no longer explicit that a group of glyphs forms a hash. We do not test whether pruning keeps the enlarged glyphs. Prompt compressors drop tokens judged uninformative [17, 18, 23, 32], learned compressors map context to soft tokens [8, 13, 31], and KV-cache methods evict entries [9, 24, 43, 51]. Selective Context, like our entropy estimate, uses an information signal,

but to *remove* low-information text. We keep every character and only change how large it is drawn. Caching prompt segments [14] is a competing way to cut cost for repeated queries that we do not evaluate.

We measure verbatim fidelity with RULER [15] and collateral damage with LongBench [3]. Text-rich benchmarks [27, 30] show the same weakness on natural images and PDFs. Modern VLMs allocate vision tokens roughly in proportion to pixel area [1, 2, 6, 21, 40], so the rendering budget is a controllable cost. We report the prompt tokens counted by each provider.

3. Information in Rendered Text

We first fix the budget (Sec. 3.1), then ask which characters need pixels (Sec. 3.2), measure how evenly a rendering serves them (Sec. 3.3), and finally flatten the resulting map (Secs. 3.4 and 3.5); Fig. 2 summarizes the pipeline.

3.1. Rendering under a budget

A document $x = (c_1, \dots, c_N)$ is a sequence of characters, grouped into whitespace-separated words. A renderer R maps x and a size assignment σ (glyph height in pixels for every word) to an image $I = R(x; \sigma)$ of fixed width, which is split into pages for the VLM. Following the patch-based accounting of dynamic-resolution VLMs [40], the vision-token cost of an image with pages of width w_p and height h_p is

$$v(I) \;=\; \sum_{p} \left\lceil \tfrac{w_p}{28} \right\rceil \left\lceil \tfrac{h_p}{28} \right\rceil , \tag{1}$$

where one vision token covers 28×28 pixels (14-pixel patches merged 2×2 in Qwen2-VL). Equation (1) is used to enforce budgets; reported costs are the prompt tokens counted by each provider. Typical COC renders every character at one base size s_0, and its cost defines the budget $B = v\big(R(x; \sigma \equiv s_0)\big)$. Every method we compare stays within B, up to page rounding, and keeps every character in place.

3.2. Which pixels matter more

Consider how a reader copies a string $c_1 \dots c_L$ from the image. As a schematic model, it combines visual evidence from each glyph, which grows with the glyph size s_i, with the expectation of its language model given the context:

$$p(c_i \mid I, \mathrm{ctx}) \;\propto\; \underbrace{p(I_i \mid c_i;\, s_i)}_{\text{pixels}}\; \underbrace{p_\theta(c_i \mid \mathrm{ctx})}_{\text{language prior}} . \tag{2}$$

The pixels must supply what the prior leaves open, roughly the *residual entropy* of each character. We estimate it from the text alone, before rendering, as the surprisal under a small causal language model q,

$$h_i \;=\; -\log_2 q(c_i \mid c_{<i}), \tag{3}$$

whose expectation is the conditional entropy $H(c_i \mid c_{<i})$ under q. We use Qwen2.5-0.5B [33] and spread each token's surprisal evenly over its characters. On our RULER documents, essay prose averages 0.9 bits per character, while the hexadecimal characters of a UUID average 3.5 bits. Exact copying fails if any character fails, so a string is only as legible as its least predictable part. When the visual evidence for a high-entropy character is too weak, the prior dominates Eq. (2). It knows the format but not the content, so we expect a well-formed string with invented characters, which is what we observe (Sec. 5.3). Under a fixed budget, pixels should therefore move from low to high residual entropy. The trade is favorable because the two kinds of text sit on different parts of their legibility curves. Found English phrases keep 92–96% character accuracy from 10 down to 4 px, while a UUID at 12 instead of 5–6 px goes from 51% to 91% hexadecimal accuracy (Sec. 5.3).

3.3. Information maps and entropy uniformity

Let a_i be the area of the glyph cell of c_i in vision tokens (pixels divided by 28^2), so that $\sum_i a_i \approx v(I)$. For a window W of n consecutive characters in reading order, the *information load*

$$\ell(W) \;=\; \frac{\sum_{i \in W} h_i}{\sum_{i \in W} a_i} \tag{4}$$

is the number of bits each of its vision tokens must carry. The page mean is $\bar{\ell} = \sum_i h_i / \sum_i a_i$. Typical COC makes a_i constant, so the load follows h_i: prose windows carry about 20 bits per vision token at 6 px and windows around a UUID about 60. We define *entropy uniformity* as

$$\mathcal{U}(I) \;=\; \frac{\bar{\ell}}{\mathrm{mean}_{W \in \mathcal{T}}\, \ell(W)}, \tag{5}$$

where $\mathcal{T}$ is the set of the 160 windows with the highest load (1% of a 16K-character document; a fixed count keeps the tail from being diluted as documents grow). $\mathcal{U} = 1$ when every window carries the same load, and $\mathcal{U}$ is small when a few windows carry much more than the rest. We use $n = 48$, a little longer than a UUID with its key, so that a single rare prose word does not form a peak on its own. Other window lengths and tail statistics give similar results (Sec. A).

Three properties make $\mathcal{U}$ useful. First, it is invariant to rescaling: multiplying every a_i by a constant changes $\bar{\ell}$ and every $\ell(W)$ by the same factor. Changing the font size of the whole page or downsampling the image therefore leaves $\mathcal{U}$ almost unchanged, and only a non-uniform allocation of pixels can raise it. Second, it is computable before any model call, from the text and the layout. A low $\mathcal{U}$ under typical COC flags hot spots that reallocation could cool, although in our data it does not predict the gain of single documents (Sec. A). Third, the load can be drawn. Coloring

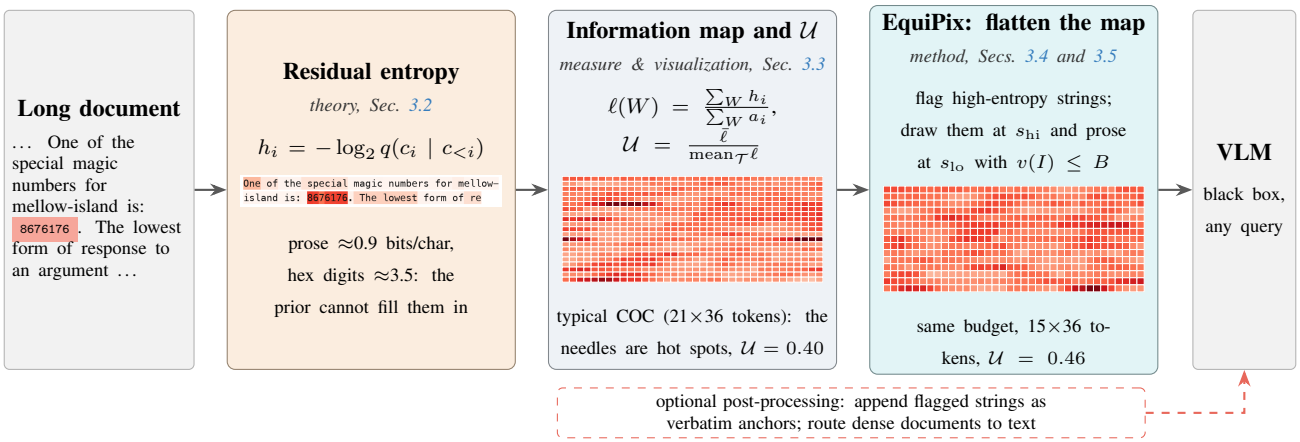


Figure 2. **From residual entropy to EquiPix.** The renderer estimates the residual entropy of every character from the text alone (left: a needle sentence colored by h_i; the key and the digits are hot). Dividing by glyph area gives the information load, which can be drawn as a map of any page and summarized by entropy uniformity $\mathcal{U}$. Typical COC leaves the high-entropy strings as hot spots. EquiPix enlarges the flagged strings and shrinks prose within the same vision-token budget, which flattens the map (same page; each cell is one vision token, as in Fig. 1). Anchors leave the map unchanged and are optional post-processing. Every step is training-free and needs no access to the model.

each vision token by its hottest window, as $\ell/(3\bar{\ell})$ clipped at 1, gives an *information map* on a 0–1 scale (Fig. 1, bottom). Its hot spots are the spans that the pixels must carry alone, and in our data they are the spans most often misread (Sec. 5.1).

$\mathcal{U}$ is a summary of the page, not of the queried string, and it measures shape at a fixed budget. Across budgets, $\bar{\ell}$ itself changes: at 10 px every window carries less than half the load it carries at 6 px.

3.4. EquiPix: flattening the map

Ideally, glyph area would be proportional to residual entropy, $a_i \propto h_i$, which makes the load constant and $\mathcal{U} = 1$. Three constraints make this impractical: glyph sizes are integers, words must stay intact, and prose cannot shrink below the size that VLMs still read. EquiPix therefore approximates the ideal with two sizes. It flags high-entropy strings with cheap text rules instead of a language model, so that rendering stays fast and model-free, and it gives the flagged strings the pixels that prose can spare.

The detector combines regular expressions with a windowed character-entropy test. After stripping punctuation, a word is flagged if it matches one of five patterns: a hexadecimal run of at least 8 characters, a run of at least 5 digits, a path with a digit or a file extension, an IP address, or a digest prefix. Words that mix letters and digits are also flagged if they have at least 20 identifier characters or a 16-character window with at least 3.5 bits of character entropy. Four-digit numbers are excluded because in natural text they are mostly years, which are cheap to read, and long English words are never flagged. Without these exclusions the detector flagged 85–112 words per LongBench document (Sec. K). The detector selects strings that the language model also finds surprising, so EquiPix raises $\mathcal{U}$ by construction. What has to be shown empirically is that higher $\mathcal{U}$ means higher recall (Sec. 5.1).

3.5. Budget-constrained glyph allocation

Given the flagged set $\mathcal{H}(x)$, prose words are drawn at s_{lo} and flagged words at s_{hi}. Prose is wrapped at character boundaries, as in typical COC, and each flagged word is kept whole on one line. The sizes are

$$(s^*_{\text{hi}}, s^*_{\text{lo}}) = \underset{\substack{s_{\text{hi}} \in [s_0, 12] \\ s_{\text{lo}} \in [5, \min(s_{\text{hi}}, s_0)]}}{\text{lexmax}} (s_{\text{hi}}, s_{\text{lo}}) \quad \text{s.t. } v\big(R(x; \sigma_{s_{\text{lo}}, s_{\text{hi}}})\big) \le B, \tag{6}$$

where lexmax first maximizes s_{hi} and then s_{lo}, because the hottest windows limit $\mathcal{U}$. The 5 px floor keeps prose readable, and $s_{\text{lo}} \le s_0$ ensures prose is never enlarged. If no pair with $s_{\text{hi}} > s_{\text{lo}}$ fits, prose-only lines lose one pixel of line spacing; at $s_0 = 5$ px this is what makes room (Sec. I). If nothing is flagged, the image is identical to typical COC. A missed high-entropy string in a document where other strings are flagged, however, shrinks with the prose. Allocation takes about 1 s for 32K and 4 s for 128K characters on one CPU core. At $s_0 = 6$ px the result usually costs 13–27% fewer tokens than the budget, because the search stops at the largest feasible s_{hi}.

4. Experimental Setup

We treat every VLM as a black box and send one stateless request per item at temperature 0 (0.01 for MiniMax), re-

sending unparseable outputs up to three times. The six readers span vendors, scales, and open weights: Claude Sonnet 5.5, Gemini 3.5 Flash-Lite, GPT-6 Luna (minimal reasoning effort), Qwen3-VL-235B-A22B [1], Llama 4 Maverick, and MiniMax-M3. Means and pooled tests use the five readers other than Claude, which was not run on every arm and is reported per reader only. Cost is the average number of prompt tokens per task as counted by each provider, including the image, the question, and any appended text. MiniMax reports no usage and is left out of cost averages only. Empty responses and transport errors (fewer than 100 of roughly 35,500 calls) are dropped; any other unparseable output counts as wrong.

Verbatim fidelity is measured with RULER needle-in-a-haystack tasks [15], re-implemented with the official templates and Paul Graham essays as the haystack. The tasks are *single* (one UUID value), *multi-key* (four keys, one queried, seven-digit values), and *multi-value* (four seven-digit values for one key). Contexts have 16K characters (Sec. 5.2), 32K–128K characters split into up to three pages (Sec. 5.4), or 32K characters at milder budgets (Sec. 5.5). Collateral damage is measured on held-out LongBench [3] MultiFieldQA-en and HotpotQA documents of at most 40K characters, drawn after the detector was frozen. We use $N = 25$ items per task and reader for RULER at 16K and 32K characters, $N = 20$ for the length study, and $N = 50$ for LongBench.

All image arms share one renderer (Menlo, 1008-pixel pages) and the budget B of typical COC at $s_0 \in \{5, 6\}$ px, about 32 and 21 characters per vision token on our long documents. Besides typical **COC**, the arms are as follows. **Font search** picks the largest uniform font and size (Menlo, Arial, Arial Narrow) that fits B, a training-free stand-in for global configuration search [7]. **Downsample** renders at 10 px and shrinks the image to B [41]. **Random words** applies our allocation to random ordinary words whose total length matches the flagged strings. **Box** uses our layout without enlargement and draws a box around each flagged string. **Half** applies our allocation to a random half of the flagged strings. As post-processing, verbatim anchors (the flagged strings copied into the prompt) are appended to COC and to EquiPix (**COC+anchor**, **EquiPix +anchor**), and **text** gives the plain text.[1]

RULER is scored by recall of the gold values (case-insensitive containment) and LongBench by token-level F1, as in the original benchmarks. Because recall does not penalize extra answers, we also report a precision-aware F1. For each reader, task, and setting (a *cell*) we compute the mean paired difference over items with a 95% bootstrap interval. Pooled effects average over cells, with intervals from a bootstrap over readers and items, and significant cells are counted after Holm correction (Sec. D).

[1] We do not compare with FocusVTC, SEER, or LensVLM, which fine-tune the reader, or with AGAR, which needs attention maps; our readers are black-box APIs.

5. Results

5.1. Does entropy uniformity predict recall?

Figure 1 (right) plots five-reader recall against $\mathcal{U}$ for the seven renderings of the 16K-character RULER documents at the 6 px budget ($\rho = 0.79$). Adding the six renderings at 5 px gives 13 arm–budget points. Over these 13 points, $\mathcal{U}$ and recall have Spearman $\rho = 0.84$ ($p = 3 \times 10^{-4}$). Random words have the lowest uniformity (0.36) and recall (0.10). COC, font search, downsampling, and the box control sit together at $\mathcal{U} = 0.385$–0.391 with recall 0.09–0.21; half enlargement reaches 0.41–0.42 and 0.43–0.47, and EquiPix 0.46–0.47 and 0.64–0.67. Over single documents the correlation is weaker ($\rho = 0.48$ over 1,075 images), because $\mathcal{U}$ summarizes a page while each question asks about one string.

Because the detector targets high-entropy strings, EquiPix raises $\mathcal{U}$ by construction, so the correlation would be circular if it rested on EquiPix alone. Three groups of points do not depend on the detector's choices. Font search and downsampling change the size of every glyph, which leaves $\mathcal{U}$ almost unchanged (Sec. 3.3). As predicted, they barely change recall (+0.04 and +0.08 at 5 px, -0.01 and -0.03 at 6 px). Random words receive the same allocation as EquiPix, but on low-entropy text. The needles shrink with the prose, the hot spots get hotter, and recall falls below COC at 6 px. Half enlargement picks the enlarged strings at random, so its two halves form a controlled comparison within the same images. Enlarged needles are recalled 74% (5 px) and 83% (6 px) of the time, the others 12% and 11%. At 6 px the non-enlarged needles do worse than under COC (0.24), because they shrink to 5 px with the prose and their load rises. Over the whole document, half enlargement lands between COC and EquiPix (+0.21 to +0.30 over COC, and 0.31 below EquiPix; Tab. 2).

The map also predicts which strings fail, without reference to any arm. For every needle we take the load of the window around it and the share of readers that recalled it (Fig. 3). Over all renderings, recall falls from 0.70 below 15 bits per vision token to 0.12 above 80 (Spearman $\rho = -0.50$, 3,760 needle–image pairs). The relation holds within COC images alone, where nothing was reallocated ($\rho = -0.36$ over 1,030 needles; 0.52 to 0.10 across the same range), and in every subset we ran. It is weakest on the natural strings ($\rho = -0.20$ and -0.25 under COC, 60 targets each, $p = 0.12$ and 0.05), whose targets the detector did not always choose.

$\mathcal{U}$ measures the shape of the load at a given budget, not its level. In every other setting (8 and 10 px, 32K–128K characters), EquiPix has both the higher $\mathcal{U}$ and the higher

Table 1. **Equal-budget comparison on RULER needle retrieval (16K characters).** Recall per reader, averaged over three NIAH tasks (N=25 each). All image columns share the vision-token budget of typical COC at 5 or 6 px. **Bold**: best within each group. Mean rows exclude Claude‡, which was not run on random words or EquiPix +anchor. Pooled over tasks and readers, EquiPix −COC is +0.55 [0.48, 0.62] at 5 px and +0.43 [0.27, 0.56] at 6 px (95% CI; Tab. 2). On these tasks the anchors contain the answer, so the anchor columns measure copying rather than reading (Sec. 5.7).

	5 px budget (≈32 chars per vision token)							6 px budget (≈21 chars per vision token)						
	equal-budget image					+ anchors (post-proc.)		equal-budget image					+ anchors (post-proc.)	
Reader	COC one size	Font search	Down-sample	Random words	EquiPix (ours)	COC +anchor	EquiPix +anchor	COC one size	Font search	Down-sample	Random words	EquiPix (ours)	COC +anchor	EquiPix +anchor
Claude Sonnet 5.5‡	0.14	0.22	0.19	–	**0.79**	**0.95**	–	0.57	0.64	0.46	–	**0.92**	**1.00**	–
Gemini 3.5 Flash-Lite	0.41	0.57	0.79	0.48	**1.00**	**1.00**	**1.00**	0.88	0.78	0.84	0.46	**1.00**	1.00	**1.00**
GPT-6 Luna	0.02	0.03	0.02	0.02	**0.50**	0.91	**0.94**	0.11	0.16	0.08	0.02	**0.65**	0.82	**0.98**
Qwen3-VL-235B	0.00	0.00	0.00	0.00	**0.55**	**1.00**	**1.00**	0.07	0.08	0.03	0.00	**0.60**	**1.00**	**1.00**
Llama 4 Maverick	0.00	0.00	0.00	0.00	**0.58**	**0.97**	**0.97**	0.00	0.01	0.00	0.00	**0.59**	0.90	**1.00**
MiniMax-M3	0.00	0.00	0.00	0.00	**0.57**	**1.00**	0.98	0.02	0.01	0.00	0.00	**0.42**	0.97	**1.00**
Mean recall	0.08	0.12	0.16	0.10	**0.64**	0.97	**0.98**	0.22	0.21	0.19	0.10	**0.65**	0.94	**1.00**
Mean F1	0.09	0.12	0.16	0.10	**0.55**	0.80	**0.80**	0.23	0.21	0.19	0.10	**0.56**	0.79	**0.83**
Avg. tokens / task	815	811	804	814	818	860	857	920	902	894	844	842	968	881

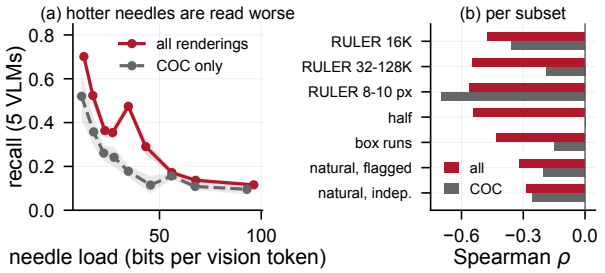


Figure 3. **The hot spots of the map are where reading fails.** (a) Recall of each needle against the absolute load of the 48-character window around it (a normalized load of 1 in Fig. 1 is 56 bits per vision token), binned (95% CI), over all renderings and settings (3,760 needle–image pairs) and over COC images only (1,030). (b) Spearman ρ between needle load and recall per subset.

recall. At 8 and 10 px, where font search was also run, $\mathcal{U}$ and recall rank the three arms identically. Across settings the mean load $\bar{\ell}$ also changes, yet pooling all 25 points still gives $\rho = 0.80$. With other window lengths and tail statistics, ρ at 16K ranges from 0.78 to 0.90 (Sec. A).

5.2. Equal-budget comparison

Table 1 compares all arms at the same vision-token budget. Averaged over five readers, typical COC recovers 8% of needles at the denser budget and 22% at the lighter one; only Claude (0.57) and Gemini (0.88) exceed 0.11 at 6 px. EquiPix raises the mean to 0.64 and 0.65, a pooled gain of 0.55 and 0.43 (Tab. 2), significant in 14 and 11 of 15 cells after Holm correction and never significantly negative. Font search reaches 0.12 and 0.21, and downsampling 0.16 and 0.19. At 6 px EquiPix also uses 8% fewer tokens per task than COC (842 vs. 920), because prose shrinks more than

Table 2. **Pooled effects** over the five non-Claude readers (recall unless noted). CI: 95% bootstrap over readers and items. Sig.: cells (reader × task) significantly positive after Holm correction within each row; neg.: significantly negative.

Comparison	Δ	95% CI	sig.	neg.
Equal budget, RULER				
EquiPix − COC, 16K, 5 px	+0.55	[+0.48, +0.62]	14/15	0
EquiPix − COC, 16K, 6 px	+0.43	[+0.27, +0.56]	11/15	0
EquiPix − COC, 32K–128K, 6 px	+0.48	[+0.43, +0.53]	35/45	0
EquiPix − font search, 6 px	+0.44	[+0.31, +0.55]	11/15	0
EquiPix − random words, 6 px	+0.56	[+0.47, +0.64]	12/15	0
box − COC, 6 px	-0.05	[-0.13, 0.00]	0/15	1
EquiPix − box, 6 px	+0.53	[+0.41, +0.64]	14/15	0
half − COC, 5 px	+0.30	[+0.21, +0.40]	9/10	0
half − COC, 6 px	+0.21	[0.00, +0.39]	8/10	0
EquiPix − half, 6 px	+0.31	[+0.20, +0.41]	6/10	0
Milder budgets (32K)				
EquiPix − COC, 10 px	+0.20	[+0.11, +0.30]	4/10	0
EquiPix − font search, 10 px	+0.04	[-0.04, +0.12]	1/10	1
Natural strings (LongBench documents)				
EquiPix − COC, flagged targets	+0.25	[+0.12, +0.38]	4/5	0
EquiPix − COC, detector-indep.	-0.01	[-0.11, +0.08]	0/5	0
Ordinary comprehension (LongBench F1)				
EquiPix − COC	+0.02	[-0.02, +0.06]	0/10	0
EquiPix +anchor − text	-0.21	[-0.29, -0.13]	0/10	9
Post-processing: EquiPix+anchor − COC+anchor (all settings: Tabs. 8 and 9)				
RULER 16K, 6 px (F1)	+0.04	[0.00, +0.08]	1/15	0
multi-key 64K–128K (F1)	-0.01	[-0.05, +0.03]	0/10	0
natural, flagged, 6 px	+0.02	[-0.06, +0.10]	0/5	0

the enlarged strings grow. The weaker readers gain most (Llama 4 Maverick from 0.00 to 0.59, GPT from 0.11 to 0.65 at 6 px), and Gemini, already at 0.88, gains 0.12. Under the precision-aware F1 the gains are smaller (0.46 and 0.34 pooled) but remain significant in at least 9 of 15 cells. Two controls isolate the ingredients. Random words, with sizes and cost close to those of EquiPix, recover 0.10 of

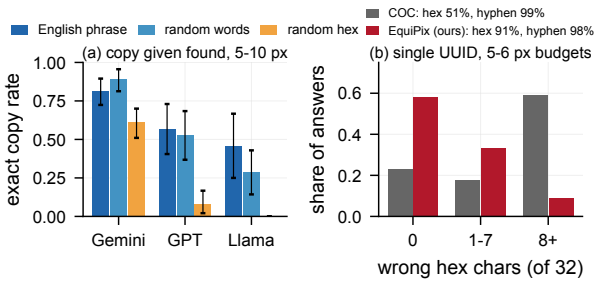


Figure 4. **Reading is limited by what the language prior cannot supply.** (a) 32-character targets of three types hidden in the same 16K-character documents and rendered by COC at 5–10 px: share copied exactly among answers that found the target (95% CI), for the three readers that found enough targets of every type. (b) Single-UUID needles at the 5 and 6 px budgets: wrong hexadecimal characters per answer of the right length. Under COC most such answers are fabricated; under EquiPix most are exact.

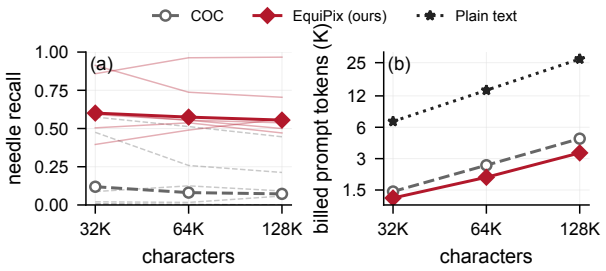


Figure 5. **Context length.** RULER at 32K–128K characters (6 px budget, up to three pages, N=20 per task). (a) Needle recall; thin lines are individual readers (including Claude), thick lines the mean of five readers. (b) Average prompt tokens per task over the four of these readers that report usage; log scale.

the needles at both budgets, 0.54 and 0.56 below EquiPix. The box control makes the needle easy to find but not easier to read. It is 0.53 below EquiPix and slightly below COC (−0.05, interval upper bound −0.001), perhaps because the box line touches the small glyphs. The gain thus comes from making the high-entropy strings legible, not from layout changes or from pointing at them.

5.3. Why does flattening help?

The account of Sec. 3.2 predicts that readers fail where the pixels must supply what the prior cannot. The single-UUID needles of Tab. 1 agree. Under COC a third of the answers are empty. In answers of the right length the four hyphens, which the format predicts, are correct 99% of the time and the 32 hexadecimal characters 51%. EquiPix raises the hexadecimal characters to 91% and leaves the hyphens at 98%, and the gain holds for every reader (from 0.15–0.96 to 0.82–1.00). Wrong answers are mostly fabricated rather than misread. Of the right-length answers, 59% have at least 8 of 32 characters wrong (median 27, close to the 30 expected from guessing), and only 23% are exact (Fig. 4b). The weaker readers drive this pattern; Gemini almost never fabricates and errs by misreading (Sec. G). Under EquiPix, 58% of the answers are exact and 9% fabricated. Among partial misreads under COC, the most frequent confusion is 8→0, two glyphs that differ by a small stroke.

To vary predictability at fixed glyph size, we hid one 32-character value in the same documents and changed only its type: an English phrase, random common words, or random hexadecimal characters, rendered by COC at 4–10 px. Three readers found enough targets of every type. For each, a found hexadecimal string is copied exactly far less often than a found English phrase: 0.61 vs. 0.82 for Gemini, 0.08 vs. 0.57 for GPT, and 0.00 vs. 0.46 for Llama (Fig. 4a). Hexadecimal and English targets also differ in their glyphs, so this test supports but does not isolate the role of the prior (Sec. G).

5.4. Scaling with context length

COC is most attractive for long documents, so Fig. 5 extends the comparison to 128K characters. Under COC, mean recall declines from 0.12 at 32K to 0.08 at 64K and 0.07 at 128K, consistent with reports that compressed reading loses retrieval ability as context grows [52]. EquiPix stays between 0.60 and 0.56, a pooled gain of 0.48 [0.43, 0.53] with 35 of 45 cells significant after Holm correction. It uses 13%, 23%, and 27% fewer tokens per task than COC at the three lengths, and 5.4×, 6.8×, and 8.0× fewer than plain text, which every reader answers with recall of at least 0.99. Readers differ widely (0.47 to 0.97 at 128K for EquiPix; Tab. 5).

5.5. Milder budgets

Our main budgets put most readers near the floor of COC, where gains are easiest. We therefore repeated single and multi-key at 32K characters with base sizes of 8 and 10 px, which cost 1.5 and 2.3 times as many vision tokens as 6 px. COC improves with size (multi-key 0.26 at 8 px and 0.60 at 10 px), and the gain of EquiPix shrinks to 0.38 at 8 px and 0.20 at 10 px. Font search catches up: EquiPix beats it by 0.26 at 8 px but only by 0.04 at 10 px, which is not significant. Single UUIDs remain hard (COC 0.20, EquiPix 0.44 at 10 px). All pooled comparisons at 5, 6, 7, and 8 px, with recall and F1, are listed in Tabs. 8 and 9 of the supplementary, and the natural identifiers at 5–7 px in Tab. 18. Reallocation helps most where the budget is tight, which is also where COC saves the most.

5.6. Natural strings and ordinary comprehension

RULER needles are synthetic, and the detector was designed with such strings in mind. We therefore built two

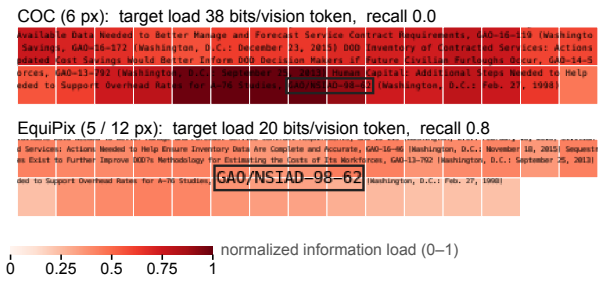


Figure 6. **Information map of a LongBench document** (GovReport, 30K characters) around the target `GAO/NSIAD-98-62` (boxed); cells as in Fig. 1. Under COC the target sits in the hottest cells and no reader copies it. EquiPix halves its load and four of five readers copy it, while the unflagged citations around it stay hot.

sets of 60 questions from LongBench documents of up to 40K characters, each asking for the exact string that follows a unique six-word phrase (Sec. M). For targets the detector flags, such as report and grant numbers, ISBNs, citation keys, and URLs, EquiPix doubles mean recall (0.23 to 0.48, significant for four of five readers) at 12% fewer tokens per task (Tab. 17). For digit-bearing targets sampled without the detector, mostly short numbers and version strings such as "25,000" or "2.0.13", only one of 60 is flagged. Every image arm is poor on these (0.36–0.45 against 0.96 for text), and EquiPix neither helps nor hurts (-0.01 [-0.11, 0.08]). A synthetic ablation agrees: needles the regex misses leave EquiPix at COC recall (0.33 vs. 0.32), whereas an oracle detector would reach 0.82 (Tab. 7).

The maps explain both results at the level of single strings. On flagged targets EquiPix halves the mean load (27.8 to 14.2 bits per vision token), while on detector-independent targets the load rises from 20.5 to 23.0 because they shrink with the prose (Fig. 6). Page-level $\mathcal{U}$, however, does not rise on these documents (0.374 to 0.365 on the flagged set). Natural documents contain many unflagged hot spots, such as names, short numbers, and citations, which grow hotter when prose shrinks. The regex detector is enough to flatten RULER pages but cools only the flagged spots of natural ones; choosing what to enlarge from the map itself is the natural next step. Shrinking prose could also hurt questions unrelated to identifiers. On held-out LongBench QA (Tab. 6), F1 changes by +0.02 [-0.02, 0.06] relative to COC, which rules out losses larger than about 0.02; one reader–task cell is significantly negative. Every image arm trails plain text: EquiPix +anchor loses 0.15–0.33 F1 in 9 of 10 cells, at a fifth to a quarter of the text cost. EquiPix removes a failure specific to identifiers but does not make COC a substitute for text.

5.7. Anchors as post-processing

Anchors append the flagged strings verbatim after the question. They leave the map unchanged, so we treat them as post-processing that can be added to either image. In the RULER essays the only flagged strings are the needles, so the anchor list is the answer and both anchor arms reach 0.90–1.00 by copying (Tab. 1). On flagged natural identifiers, where documents carry 5.3 anchors on average, EquiPix +anchor lies on or above the recall–cost frontier of COC+anchor by 0.02–0.05 across 5–7 px, though no pooled interval excludes zero. At 64K–128K characters it costs 29–31% less (Sec. H). When the target is not flagged, anchors can mislead (0.36 vs. 0.44 without them). Beyond about 16 flagged strings per 32K characters, anchors on either image become expensive and routing to text takes over (Sec. L).

6. Limitations and Conclusion

Our residual-entropy estimate is the surprisal of a small left-to-right language model. It ignores the right-hand context and the visual distinctiveness of glyphs, which is why we smooth it over windows and use $\mathcal{U}$ to compare renderings of the same documents rather than as an absolute score. The correlation with recall rests on 13 arm–budget points from synthetic needles, and EquiPix raises $\mathcal{U}$ partly by construction. On natural documents the regex detector cools the flagged targets but leaves page-level $\mathcal{U}$ unchanged. The gains are largest at dense budgets where COC fails; at 10 px font search does as well. COC, with or without reallocation, trails plain text on ordinary QA by up to 0.33 F1. Our natural sets are small (60 automatically constructed questions each), the detector protects only high-entropy strings, and anchors can mislead when the target is not flagged. We test black-box readers through public APIs whose versions and preprocessing may change, and we do not test trained optical compressors [7, 41] or token pruning. Budgets follow one patch-based rule (Eq. (1)), and token savings have exceptions at short lengths (Gemini up to 32K characters, Qwen at 16K and 5 px, and Llama, whose usage ignores image size). We use ASCII English only and do not measure latency, prompt caching [14], or whether text inside images bypasses text-side safety filters.

A rendered page does not carry information evenly. The renderer can see this before any model call: residual entropy, estimated from the text, gives an information map of the page, and entropy uniformity summarizes it. At an equal budget, renderings with a flatter map are read better, and renderings that only rescale the page leave both unchanged. EquiPix flattens the map by moving pixels from prose to high-entropy strings. Across five readers it turns fabricated identifiers into read ones, raises needle recall by about half the scale at dense budgets, and doubles the recall of flagged natural identifiers. Plain text still wins on ordinary QA.

References

[1] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, et al. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025. 3, 5

[2] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, et al. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025. 3

[3] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 3119–3137, 2024. 3, 5

[4] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your ViT but faster. In *Proceedings of the International Conference on Learning Representations*, 2023. 2

[5] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *Proceedings of the European Conference on Computer Vision*, pages 19–35, 2024. 2

[6] Zhe Chen, Weiyun Wang, Hao Tian, Shenglong Ye, Zhangwei Gao, Erfei Cui, Wenwen Tong, Kongzhi Hu, et al. How far are we to GPT-4V? closing the gap to commercial multimodal models with open-source suites. *Science China Information Sciences*, 67(12):220101, 2024. 3

[7] Jiale Cheng, Yusen Liu, Xinyu Zhang, Yulin Fei, Wenyi Hong, Ruiliang Lyu, Weihan Wang, Zhe Su, Xiaotao Gu, Xiao Liu, et al. Glyph: Scaling context windows via visual-text compression. *arXiv preprint arXiv:2510.17800*, 2025. 1, 2, 5, 8

[8] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3829–3846, 2023. 2

[9] Alessio Devoto, Maximilian Jeblick, and Simon Jégou. Expected attention: KV cache compression by estimating attention from future queries distribution. *arXiv preprint arXiv:2510.00636*, 2025. 2

[10] Zhengyao Fang, Pengyuan Lyu, Chengquan Zhang, Guangming Lu, Jun Yu, and Wenjie Pei. Prune redundancy, preserve essence: Vision token compression in VLMs via synergistic importance-diversity. In *Proceedings of the International Conference on Learning Representations*, 2026. 2

[11] Tianyu Gao, Zirui Wang, Adithya Bhaskar, and Danqi Chen. Improving language understanding from screenshots. *arXiv preprint arXiv:2402.14073*, 2024. 2

[12] Yonghan Gao, Zehong Chen, Lijian Xu, Jingzhi Chen, Jingwei Guan, and Xingyu Zeng. ZeroSense: How vision matters in long context compression. *arXiv preprint arXiv:2603.11846*, 2026. 2

[13] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context autoencoder for context compression in a large language model. In *Proceedings of the International Conference on Learning Representations*, 2024. 2

[14] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024. 3, 8

[15] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? In *Proceedings of the First Conference on Language Modeling*, 2024. 3, 5

[16] Anwen Hu, Haiyang Xu, Liang Zhang, Jiabo Ye, Ming Yan, Ji Zhang, Qin Jin, Fei Huang, and Jingren Zhou. mPLUG-DocOwl2: High-resolution compressing for OCR-free multi-page document understanding. *arXiv preprint arXiv:2409.03420*, 2024. 2

[17] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376, 2023. 2

[18] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 1658–1677, 2024. 2

[19] Ivan Yee Lee, Cheng Yang, and Taylor Berg-Kirkpatrick. Optical context compression is just (bad) autoencoding. In *Proceedings of the Third Conference on Language Modeling*, 2026. 2

[20] Kenton Lee, Mandar Joshi, Iulia Raluca Turc, Hexiang Hu, Fangyu Liu, Julian Martin Eisenschlos, Urvashi Khandelwal, Peter Shaw, Ming-Wei Chang, and Kristina Toutanova. Pix2Struct: Screenshot parsing as pretraining for visual language understanding. In *Proceedings of the 40th International Conference on Machine Learning*, pages 18893–18912, 2023. 2

[21] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Peiyuan Zhang, Yanwei Li, Ziwei Liu, and Chunyuan Li. LLaVA-OneVision: Easy visual task transfer. *Transactions on Machine Learning Research*, 2025. 3

[22] Jianjian Li, Junquan Fan, Feng Tang, Gang Huang, Shitao Zhu, Songlin Liu, Nian Xie, Wulong Liu, and Yong Liao. FCoT-VL: Advancing text-oriented large vision-language models with efficient visual token compression. *arXiv preprint arXiv:2502.18512*, 2025. 2

[23] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6342–6353, 2023. 2

[24] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural*

Information Processing Systems, pages 22947–22970, 2024. 2

[25] Yanhong Li, Zixuan Lan, and Jiawei Zhou. Text or pixels? evaluating efficiency and understanding of LLMs with visual text inputs. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 10564–10578, 2025. 2

[26] Yunhao Liang, Ruixuan Ying, Bo Li, Hong Li, Kai Yan, Qingwen Li, Min Yang, Satoshi Okamoto, Zhe Cui, and Shiwen Ni. Visual merit or linguistic crutch? a close look at DeepSeek-OCR. *arXiv preprint arXiv:2601.03714*, 2026. 2

[27] Yuliang Liu, Zhang Li, Mingxin Huang, Biao Yang, Wenwen Yu, Chunyuan Li, Xucheng Yin, Cheng-lin Liu, Lianwen Jin, and Xiang Bai. OCRBench: On the hidden mystery of OCR in large multimodal models. *Science China Information Sciences*, 67(12):220102, 2024. 3

[28] Yuliang Liu, Biao Yang, Qiang Liu, Zhang Li, Zhiyin Ma, Shuo Zhang, and Xiang Bai. TextMonkey: An OCR-free large multimodal model for understanding document. *arXiv preprint arXiv:2403.04473*, 2024. 2

[29] Guangchuan Lv, Dianxing Shi, and Dingjie Fu. VPRune: Efficient training-free pre-LLM visual token pruning. *arXiv preprint arXiv:2609.24485*, 2026. 2

[30] Yubo Ma, Yuhang Zang, Liangyu Chen, Meiqi Chen, Yizhu Jiao, Xinze Li, Xinyuan Lu, Ziyu Liu, Yan Ma, Xiaoyi Dong, et al. MMLongBench-Doc: Benchmarking long-context document understanding with visualizations. In *Advances in Neural Information Processing Systems*, pages 95963–96010, 2024. 3

[31] Jesse Mu, Xiang Lisa Li, and Noah Goodman. Learning to compress prompts with gist tokens. In *Advances in Neural Information Processing Systems*, pages 19327–19352, 2023. 2

[32] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. 2

[33] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. 3

[34] Phillip Rust, Jonas F. Lotz, Emanuele Bugliarello, Elizabeth Salesky, Miryam de Lhoneux, and Desmond Elliott. Language modelling with pixels. In *Proceedings of the International Conference on Learning Representations*, 2023. 1, 2

[35] Yuzhang Shang, Mu Cai, Bingxin Xu, Yong Jae Lee, and Yan Yan. LLaVA-PruMerge: Adaptive token reduction for efficient large multimodal models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 22857–22867, 2025. 2

[36] Kaiser Sun, Xiaochuang Yuan, Hongjun Liu, Chen Zhao, Cheng Zhang, Mark Dredze, and Fan Bai. Reading, not thinking: Understanding and bridging the modality gap when text becomes pixels in multimodal LLMs. *arXiv preprint arXiv:2603.09095*, 2026. 2

[37] Michael Tschannen, Basil Mustafa, and Neil Houlsby. CLIPPO: Image-and-language understanding from pixels only. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11006–11017, 2023. 2

[38] Ben Wan, Yan Feng, Zihan Tang, Weizhe Huang, Yuting Zeng, Jia Wang, and Tongxuan Liu. RTPrune: Reading-twice inspired token pruning for efficient DeepSeek-OCR inference. In *Proceedings of the 43rd International Conference on Machine Learning*, pages 124471–124491, 2026. 2

[39] Alex Jinpeng Wang, Linjie Li, Yiqi Lin, Min Li, Lijuan Wang, and Mike Zheng Shou. Leveraging visual tokens for extended text contexts in multi-modal learning. In *Advances in Neural Information Processing Systems*, pages 14325–14348, 2024. 2

[40] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, et al. Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024. 3

[41] Haoran Wei, Yaofeng Sun, and Yukun Li. DeepSeek-OCR: Contexts optical compression. *arXiv preprint arXiv:2510.18234*, 2025. 1, 2, 5, 8

[42] Haoran Wei, Yaofeng Sun, and Yukun Li. DeepSeek-OCR 2: Visual causal flow. *arXiv preprint arXiv:2601.20552*, 2026. 2

[43] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *Proceedings of the International Conference on Learning Representations*, 2024. 2

[44] Roy Xie, Dan Friedman, Donghan Yu, Bowen Pan, Christopher Fifty, Jang-Hyun Kim, Xianzhi Du, Zhe Gan, Vivek Rathod, and Bhuwan Dhingra. LensVLM: Selective context expansion for compressed visual representation of text. *arXiv preprint arXiv:2605.07019*, 2026. 2

[45] Ling Xing, Alex Jinpeng Wang, Rui Yan, Xiangbo Shu, and Jinhui Tang. Vision-centric token compression in large language model. In *Advances in Neural Information Processing Systems*, pages 33080–33110, 2025. 2

[46] Jiawei Xu, Zhilin Zhai, Jinrui Fang, Ruohan Xu, Mingfei Lu, Yi Zhang, Guanchu Wang, Tianlong Chen, and Ying Ding. SEER: Long-context reasoning via selective visual-text compression. In *Proceedings of the Third Conference on Language Modeling*, 2026. 2

[47] Senqiao Yang, Yukang Chen, Zhuotao Tian, Chengyao Wang, Jingyao Li, Bei Yu, and Jiaya Jia. VisionZip: Longer is better but not necessary in vision language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19792–19802, 2025. 2

[48] Shenglai Zeng, Qirui Wang, Kai Guo, Xinnan Dai, Xianxuan Long, and Hui Liu. Magnifying what matters: Attention-guided adaptive rendering for visual text comprehension. *arXiv preprint arXiv:2606.12898*, 2026. 2

[49] Haokui Zhang, Congyang Ou, Dawei Yan, Peng Wang, Qingsen Yan, Yu Zhang, Ying Li, and Rong Xiao. TRIO: Token reduction via inference-objective guidance for efficient vision-language models. *arXiv preprint arXiv:2602.04657*, 2026. 2

[50] Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis A. Gudovskiy, Tomoyuki

Okuno, Yohei Nakata, Kurt Keutzer, and Shanghang Zhang. SparseVLM: Visual token sparsification for efficient vision-language model inference. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 74840–74857, 2025. 2

[51] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, pages 34661–34710, 2023. 2

[52] Hongbo Zhao, Meng Wang, Fei Zhu, Wenzhuo Liu, Bolin Ni, Fanhu Zeng, Gaofeng Meng, and Zhaoxiang Zhang. VTCBench: Can vision-language models understand long context with vision-text compression? *arXiv preprint arXiv:2512.15649*, 2025. 2, 7

[53] FangZhi Zhong, Xuerui Qiu, Yuqi Pan, Ya Liu, Shaowei Gu, Bo Xu, and Guoqi Li. FocusVTC: Efficient and high-performance visual text compression with adaptive resolution. *arXiv preprint arXiv:2609.36651*, 2026. 2

LLM Usage Statement

Large language models are the *objects of study* in this work: the six vision-language models listed in Sec. 4 and the additional Claude reader of Sec. O were queried as black-box readers, and all reported numbers are computed by our scripts from their logged outputs. Every number in the paper is produced programmatically from the experiment logs, and every reference was checked against its primary source. No LLM was used to generate or label evaluation data beyond the readers' answers being evaluated.

Which Pixels Matter More? Entropy-Uniform Rendering for Optical Long-Context Compression

Supplementary Material

Notation

The supplementary tables keep the one-letter arm codes of our experiment logs: A = typical COC (one size), U = font search, Z = downsample, R = random words, B = box, H = half enlargement, S = EquiPix, K = COC+anchor, X = EquiPix +anchor, T = plain text. All costs are average prompt tokens per task as counted by each provider.

A. Entropy Uniformity: Details

Residual entropy (Eq. (3)) is computed with Qwen2.5-0.5B in windows of 1024 tokens with a stride of 512, so that every token is scored with at least 512 tokens of context. Each token's surprisal is divided evenly among its characters. Glyph cells are recomputed from the layout of each image (font, sizes, line spacing, and the downsampling factor), and the recomputed image heights match the stored images for every image we checked. For each image we compute the load of every 48-character window and $\mathcal{U}$ as in Eq. (5).

Table 3 lists $\mathcal{U}$, the mean load $\bar{\ell}$, and recall for every arm and setting. Within a setting, COC, font search, downsampling, and the box control differ in $\mathcal{U}$ by at most 0.01, as expected from the rescaling invariance. EquiPix raises $\bar{\ell}$ because the image is smaller than the budget, and it raises $\mathcal{U}$ because the hottest windows cool down. Random words raise $\bar{\ell}$ as well but lower $\mathcal{U}$, because the needles shrink with the prose. Table 4 varies the window length and the tail statistic. At 16K characters every variant gives ρ between 0.78 and 0.90. Across lengths, the top-1% variant weakens the correlation (Tab. 4), because its tail grows with the document and fills with prose windows; this motivated the fixed count of 160 windows. We chose $n = 48$ before running the robustness analysis, but we chose the fixed-count tail after seeing the length results, so the pooled ρ over all settings should be read with this in mind.

Whether $\mathcal{U}$ can triage documents is a separate question. For each document we compared $\mathcal{U}$ under COC with the gain of EquiPix over COC, averaged over readers. Within the RULER and natural-identifier sets the rank correlation is at most 0.18 in magnitude. Inside one set the documents have similar hot spots, so the gain depends mostly on whether the queried string is flagged, which $\mathcal{U}$ does not see. We therefore use $\mathcal{U}$ to compare renderings of the same documents, not to predict the gain on a single document.

B. Implementation Details

Renderer. Pages are 1008 pixels wide (36 patches) with a 4-pixel margin and are rendered in grayscale with the Menlo monospace font through PIL. Prose is wrapped at character boundaries; a flagged word is placed whole on the next line when it does not fit. Each line is as tall as its largest glyph plus one pixel of leading. Documents taller than 2016 pixels are split into pages at blank pixel rows, so no line is cut. All pages are sent in one request in reading order. The budget of Eq. (1) is enforced on the unsplit image. Splitting can add one patch row per page, which made 10 of the 180 reallocated long-context images 1.2% more expensive than their COC counterparts. All reported costs are the prompt tokens counted by each provider and include this effect.

Detector. After stripping the characters `.,;:()[]"'` from both ends, a word is flagged if it matches `[0-9a-f]{8,}`, `\d{5,}`, `/\S*\d\S*`, `\d+\.\d+\.\d+\.\d+`, `\S+/\S+\.\w{1,5}`, or `sha256=`. A word with at least one letter and one digit is also flagged if it contains `[A-Za-z0-9_\-]{20,}`. It is flagged as well if it is at least 16 characters long and has a 16-character window whose character entropy is at least 3.5 bits.

Baselines. U tries Menlo, Arial, and Arial Narrow at sizes 14 down to 5 px and keeps the largest size that fits the budget (ties broken by font order). At the 5 px and 6 px budgets it selects 6 px Arial and 8 px Arial Narrow for most documents. Z renders at 10 px and shrinks the image by repeated factors of 0.98 with Lanczos resampling until it fits the budget (final scale 0.52–0.64). For RULER and the natural-identifier questions, K and X append the line "Exact strings copied verbatim from the text block in the image:" after the question, followed by one flagged string per line. For LongBench, the anchors are inserted after the image placeholder and before the question, under "Exact strings copied verbatim from the text in the image:".

Table 3. **Entropy uniformity and recall for every arm and setting.** $\mathcal{U}$ (Eq. (5), n=48, top 160 windows), mean load $\bar{\ell}$ in bits per vision token, and recall averaged over the five non-Claude readers. Within each setting all arms share one budget.

Setting	Arm	$\mathcal{U}$	$\bar{\ell}$	Recall
16K, 5 px	COC	0.386	29.2	0.09
	font search	0.391	30.4	0.12
	downsample	0.386	30.2	0.17
	random words	0.356	29.7	0.10
	half	0.417	31.4	0.43
	EquiPix	0.460	30.7	0.64
16K, 6 px	COC	0.389	19.5	0.21
	font search	0.391	21.4	0.21
	downsample	0.386	20.8	0.19
	box	0.391	19.5	0.15
	random words	0.361	26.1	0.10
	half	0.412	28.0	0.47
	EquiPix	0.466	27.4	0.67
32K, 8 px	COC	0.363	12.5	0.15
	font search	0.374	13.3	0.26
	EquiPix	0.444	12.8	0.53
32K, 10 px	COC	0.363	8.2	0.40
	font search	0.373	8.7	0.56
	EquiPix	0.410	9.2	0.60
32K, 6 px	COC	0.365	19.2	0.12
	EquiPix	0.448	25.2	0.60
64K, 6 px	COC	0.351	19.2	0.08
	EquiPix	0.419	26.2	0.57
128K, 6 px	COC	0.343	19.3	0.07
	EquiPix	0.394	28.5	0.55

Table 4. **Robustness of the recall–uniformity correlation** to the window length n and the tail statistic. Spearman ρ over the 13 arm–budget points at 16K characters (equal budget), over all 25 points, and over single images at 16K (reader-averaged recall).

Tail statistic	n	ρ (16K points)	ρ (all points)	ρ (images)
top 160 windows	16	0.86	0.64	0.35
top 160 windows	32	0.84	0.74	0.45
top 160 windows	48	0.84	0.80	0.48
top 160 windows	64	0.86	0.80	0.51
top 1% of windows	16	0.86	0.54	0.35
top 1% of windows	32	0.84	0.58	0.45
top 1% of windows	48	0.84	0.61	0.48
top 1% of windows	64	0.86	0.60	0.51
maximum window	16	0.78	0.77	0.26
maximum window	32	0.90	0.87	0.40
maximum window	48	0.86	0.78	0.53
maximum window	64	0.86	0.75	0.52

Prompts and scoring. RULER prompts follow the official template, with the context replaced by "[The text is rendered in the image above.]". They add "Copy each one exactly, character for character." and ask for a JSON list of values. A gold value counts as recalled if it appears (case-insensitively) in any returned string. LongBench prompts are the official ones for each task, and the first output line is scored with the official QA F1. For the plain-text arm the context is inserted verbatim into the same template.

Router threshold. τ is chosen per reader by two-fold cross-validation on the density sweep (folds by item parity). The criterion is recall minus 0.5 times the cost relative to plain text, maximized over $\tau \in \{0, 0.05, 0.1, 0.2, 0.3, 0.5, 0.75, 1, 1.5, 2, 4, \infty\}$. Selected values were 0.1–0.5 with median 0.3, which we use for all other experiments. The frontier in Fig. 9c sweeps τ, and the per-density figures use the median τ. Both are therefore in-sample, unlike the cross-validated aggregate.

C. Per-Reader Results

Tables 5 and 22 give per-reader numbers for Figs. 5 and 9, and Tab. 7 gives the detector ablation of Sec. 5.6. In the length study, S is above A for every reader at every length. The size of the gain depends mostly on how well a reader does under COC. Claude and Gemini start at 0.45–0.57 and 0.21–0.47 under A, and S lifts them to 0.86–0.97 and 0.70–0.91. GPT, Qwen3-VL, Llama, and MiniMax recover at most 0.12 under A at every length, and S lifts them to 0.40–0.60. With length, S rises for Claude and MiniMax and falls for the other readers. Gemini falls most (0.91 to 0.70), and its COC curve falls further (0.47 to 0.21). In the detector ablation, the regex and oracle detectors give the same recall when the needles are seven-digit numbers. With four-digit needles, which the regex detector deliberately ignores, the regex arms match COC while the oracle arms reach 0.82 (S) and 0.99 (X).

Table 5. Per-reader recall for the length study (Fig. 5). The mean excludes Claude.

	32K		64K		128K	
Reader	A	S	A	S	A	S
Claude Sonnet 5.5	0.57	**0.86**	0.51	**0.96**	0.45	**0.97**
Gemini 3.5 Flash-Lite	0.47	**0.91**	0.26	**0.74**	0.21	**0.70**
GPT-6 Luna	0.09	**0.59**	0.12	**0.55**	0.09	**0.50**
Qwen3-VL-235B	0.02	**0.60**	0.02	**0.55**	0.06	**0.54**
Llama 4 Maverick	0.01	**0.50**	0.00	**0.54**	0.00	**0.47**
MiniMax-M3	0.00	**0.40**	0.00	**0.49**	0.00	**0.56**
Mean	0.12	0.60	0.08	0.57	0.07	0.56

D. Statistical Procedures

Metrics. Recall is the fraction of gold values that appear, case-insensitively, as a substring of some returned value. The precision-aware F1 deduplicates the returned values, counts a returned value as correct when it contains a gold value, and combines this precision with recall. An empty answer scores zero. Exact-set accuracy requires every gold value to be found and no other value to be returned. Outputs that cannot be parsed are scored as empty answers. Empty responses and transport errors are dropped (Sec. 4).

Per-cell tests. For one reader on one task at one setting, we bootstrap the mean paired difference over items. The per-cell intervals in the tables use 10,000 resamples, and the p-values used for Holm correction use 1,000. A cell is significant when the two-sided bootstrap p-value is below 0.05.

Pooled effects. The pooled effect is the mean paired difference over all (run, reader, item) triples of a family, where a family is one row of Tab. 2. The item-level interval resamples items within each run, with the same item draw for every reader, so it treats the five readers as fixed. The reader-and-item interval also resamples readers with replacement. Because readers are crossed with runs, one reader draw is shared by all runs of the family. Both use 4,000 resamples and percentile intervals. Within each family we apply Holm's step-down correction to the per-cell p-values and report how many cells

Table 6. **Ordinary comprehension.** LongBench QA F1 on held-out samples (N=50 per task, contexts ≤40K characters, 6 px budget, five readers). */†: paired 95% CI vs. COC above/below zero. Relative to COC, EquiPix +anchor has no significantly negative cell and EquiPix one; every image arm trails plain text (EquiPix +anchor − text significantly negative in 9 of 10 cells). Avg. tokens / task: four readers that report usage.

Reader	MultiFieldQA-en					HotpotQA				
	COC	EquiPix	COC+anchor	EquiPix +anchor	+Text	COC	EquiPix	COC+anchor	EquiPix +anchor	+Text
Gemini 3.5 Flash-Lite	0.47	0.47	0.53	0.47	0.63	0.59	0.62	0.58	0.57	0.78
GPT-6 Luna	0.36	0.39	0.37	0.36	0.56	0.68	0.73	0.68	0.70	0.72
Qwen3-VL-235B	0.27	0.26	0.31	0.26	0.59	0.66	0.61†	0.65	0.62	0.77
Llama 4 Maverick	0.28	0.25	0.34	0.29	0.59	0.45	0.50	0.51*	0.50	0.68
MiniMax-M3	0.23	0.26	0.25	0.25	0.56	0.29	0.34	0.41*	0.42*	0.69
Mean	0.32	0.33	0.36	0.33	0.58	0.53	0.56	0.57	0.56	0.73
Avg. tokens / task	1141	1056	1440	1355	5342	1200	1114	1239	1153	5617

Table 7. **Detector ablation.** RULER multi-value recall (16K characters, 6 px budget, N=25, mean of 5 readers; Claude was not run in this ablation). The regex detector flags seven-digit values but deliberately skips four-digit ones, which collide with years. On four-digit values the regex arms therefore fall back to uniform rendering.

Arm	7-digit values (detected)	4-digit values (missed)
Uniform (A)	0.22	0.32
Ours, regex detector (S)	0.77	0.33
Ours, oracle detector	0.78	0.82
Ours + anchors, regex (X)	0.99	0.32
Ours + anchors, oracle	0.99	0.99

remain significant in each direction. Table 8 lists every comparison we computed, including those not discussed in the main text.

E. Precision-Aware Scoring and Anchor Content

For every RULER setting with anchors, Tab. 10 reports recall and precision-aware F1 for A, S, K, and X, and how many values each arm returns. It also reports two baselines that never look at the image: returning every anchor (dump) and returning as many random anchors as there are gold values (pick). Three observations qualify the anchor results in the main text. First, on single and multi-value the anchors are exactly the needles, so K and X match the dump baseline and their recall says nothing about reading. Second, on multi-key and in the density sweep at $k \geq 4$, several readers (Qwen3-VL and MiniMax most often) return most or all of the anchor list. Recall then stays high while F1 falls to the level of the pick baseline. Third, COC sometimes produces long lists of hallucinated identifiers (up to 55 per item for MiniMax at 5 px), which F1 penalizes and recall does not. The gains of S over A are therefore smaller under F1 but remain significant (Tab. 8). On the natural-identifier questions, readers return one value per question for every arm, so recall and F1 coincide.

Table 11 gives the rate of well-formed near-misses on RULER. A gold value counts as a near-miss when it was not recalled but some returned string has the same length and at least 60% character similarity. EquiPix does not reduce this rate. On average it rises from 0.12 to 0.15 at 5 px and from 0.08 to 0.15 at 6 px. A plausible cause is that EquiPix turns many complete misses into nearly correct readings, some of which are still wrong. Anchors nearly remove near-misses where they contain the answer. We therefore do not claim that EquiPix makes errors detectable.

F. Controls, Milder Budgets, and Replication

Table 12 gives per-reader results for the box control and the milder budgets of Secs. 5.2 and 5.5, and Fig. 7 shows one item from the box runs. The box runs used a fresh sample of 25 items per task and re-read A and S, so they also replicate Tab. 1. On the new items, A averages 0.20 and S 0.68, against 0.22 and 0.65 on the original items at the same budget. At 8 and 10 px the U arm switches to Arial Narrow at 10 and 13 px, so it catches up with S at the lighter budget. Anchors (K, X) reach

Table 8. **All pooled comparisons, RULER at 16K characters** (five non-Claude readers; recall and precision-aware F1). CI_{item} resamples items within each run with the readers fixed. $CI_{rd+item}$ also resamples readers (one draw shared by all runs). Cells are counted as significantly positive/negative by per-cell bootstrap, raw and after Holm correction within the row.

Setting	Comparison	Metric	Δ	CI_{item}	$CI_{rd+item}$	raw +/−	Holm +/−	cells
RULER 16K, 5 px	S−A	recall	+0.55	[+0.50, +0.60]	[+0.48, +0.62]	14/0	14/0	15
		F1	+0.46	[+0.41, +0.50]	[+0.39, +0.54]	14/0	14/0	15
RULER 16K, 5 px	S−U	recall	+0.52	[+0.47, +0.56]	[+0.44, +0.60]	14/0	13/0	15
		F1	+0.42	[+0.38, +0.46]	[+0.37, +0.47]	13/0	13/0	15
RULER 16K, 5 px	S−Z	recall	+0.47	[+0.43, +0.52]	[+0.34, +0.59]	13/0	13/0	15
		F1	+0.38	[+0.34, +0.42]	[+0.29, +0.46]	13/0	13/0	15
RULER 16K, 5 px	S−R	recall	+0.54	[+0.49, +0.58]	[+0.47, +0.60]	14/0	14/0	15
		F1	+0.45	[+0.41, +0.48]	[+0.39, +0.50]	14/0	14/0	15
RULER 16K, 5 px	X−K	recall	+0.01	[-0.01, +0.02]	[-0.02, +0.04]	0/0	0/0	15
		F1	0.00	[-0.01, +0.02]	[-0.02, +0.03]	0/0	0/0	15
RULER 16K, 6 px	S−A	recall	+0.43	[+0.39, +0.47]	[+0.27, +0.56]	11/0	11/0	15
		F1	+0.34	[+0.30, +0.37]	[+0.21, +0.45]	11/0	9/0	15
RULER 16K, 6 px	S−U	recall	+0.44	[+0.40, +0.48]	[+0.31, +0.55]	11/0	11/0	15
		F1	+0.35	[+0.31, +0.38]	[+0.26, +0.43]	11/0	10/0	15
RULER 16K, 6 px	S−Z	recall	+0.47	[+0.42, +0.51]	[+0.30, +0.60]	11/0	11/0	15
		F1	+0.38	[+0.34, +0.41]	[+0.24, +0.49]	11/0	11/0	15
RULER 16K, 6 px	S−R	recall	+0.56	[+0.51, +0.60]	[+0.47, +0.64]	12/0	12/0	15
		F1	+0.47	[+0.43, +0.50]	[+0.38, +0.55]	12/0	12/0	15
RULER 16K, 6 px	X−K	recall	+0.05	[+0.03, +0.08]	[0.00, +0.12]	3/0	1/0	15
		F1	+0.04	[+0.01, +0.06]	[0.00, +0.08]	2/0	1/0	15
RULER 16K, 6 px (box runs)	B−A	recall	-0.05	[-0.07, -0.03]	[-0.13, 0.00]	0/3	0/1	15
		F1	-0.05	[-0.07, -0.03]	[-0.13, 0.00]	0/3	0/1	15
RULER 16K, 6 px (box runs)	S−B	recall	+0.53	[+0.50, +0.57]	[+0.41, +0.64]	14/0	14/0	15
		F1	+0.45	[+0.42, +0.49]	[+0.36, +0.56]	14/0	14/0	15
RULER 16K, 6 px (box runs)	S−A	recall	+0.49	[+0.45, +0.52]	[+0.29, +0.63]	13/0	13/0	15
		F1	+0.40	[+0.37, +0.44]	[+0.26, +0.53]	14/0	14/0	15

0.96–1.00 at both budgets because the anchors again contain the needles.

G. Why Flattening Helps: Details

Anatomy of UUID answers. Table 13 pools the single-UUID needles of the RULER 16K runs at 5 and 6 px with the box runs, and Tab. 14 splits them by reader. Under COC, a third of the answers are empty. The answers of the right length split mainly into exact copies (23%) and fabrications (59% with at least 8 of 32 hexadecimal characters wrong, median 27). Partial misreads with 1–7 wrong characters are the smallest group. The hyphens are correct in 99% of the right-length answers, so the readers reproduce the UUID format even when they invent the content. The pooled pattern hides large reader differences. All right-length COC answers of Llama and MiniMax are fabricated, as are most of those of Qwen3-VL and GPT, although these two return few right-length answers. Gemini fabricates in only 1% and reaches 0.96 hexadecimal accuracy. EquiPix raises hexadecimal accuracy for every reader and cuts fabrication to 0–19%. Over all right-length COC answers, fabrications included, the most frequent substitutions are 8→0, 0→8, 8→b, 6→d, 8→9, and 3→1. Among partial misreads alone (89 wrong characters), 8→0 dominates (24), followed by 1→i (6).

Entropy × glyph-size study. Each of 20 RULER documents (16K characters of essay text) receives one needle, "One of the special magic codes for *key* is: *value*.", at the same position and with the same key for every condition. The value is a 32-character string of one of three types. English phrases join consecutive all-lowercase words from LongBench HotpotQA passages, skipping capitalized words and numbers, so they are fluent but not always contiguous spans. Random words are drawn from the 2,000 most frequent lowercase words of the essays, and random hexadecimal strings use lowercase characters. Each of the 60 documents is rendered by COC at 4, 5, 6, 7, 8, and 10 px and read by the five readers with a prompt that asks for the code character for character. A target counts as found if some answer has a similarity ratio of at least 0.5 to it (random hexadecimal pairs average 0.24 and never reach 0.5). Exact copy is case-insensitive containment. Character accuracy is the similarity ratio of found answers, so its floor is 0.5. MiniMax and Qwen3-VL usually answered English and word targets with the key. Their found rates are therefore not comparable across types, and we exclude these two readers from the transcription comparison (Tabs. 15 and 16). For Gemini, GPT, and Llama, a found hexadecimal string is copied

Table 9. **All pooled comparisons, other settings** (five non-Claude readers). On natural strings, recall and F1 coincide. On LongBench, only F1 is defined. $\mathrm{CI}_{\mathrm{item}}$ resamples items within each run with the readers fixed. $\mathrm{CI}_{\mathrm{rd+item}}$ also resamples readers (one draw shared by all runs). Cells are counted as significantly positive/negative by per-cell bootstrap, raw and after Holm correction within the row.

Setting	Comparison	Metric	Δ	$\mathrm{CI}_{\mathrm{item}}$	$\mathrm{CI}_{\mathrm{rd+item}}$	raw $+/-$	Holm $+/-$	cells
RULER 32K–128K, 6 px	S−A	recall	+0.48	[+0.46, +0.51]	[+0.43, +0.53]	39/0	35/0	45
		F1	+0.38	[+0.35, +0.40]	[+0.34, +0.41]	35/0	30/0	45
natural, flagged, 5 px	X−K	recall	+0.04	[-0.01, +0.09]	[-0.03, +0.11]	0/0	0/0	5
natural, flagged, 5 px	X−K	F1	+0.03	[-0.02, +0.08]	[-0.03, +0.10]	0/0	0/0	5
natural, flagged, 6 px	S−A	recall	+0.25	[+0.18, +0.31]	[+0.12, +0.38]	4/0	4/0	5
natural, flagged, 6 px	S−A	F1	+0.25	[+0.18, +0.31]	[+0.12, +0.38]	4/0	4/0	5
natural, flagged, 6 px	X−A	recall	+0.52	[+0.45, +0.60]	[+0.33, +0.68]	5/0	5/0	5
natural, flagged, 6 px	X−A	F1	+0.52	[+0.44, +0.59]	[+0.33, +0.67]	5/0	5/0	5
natural, flagged, 6 px	X−K	recall	+0.02	[-0.04, +0.08]	[-0.06, +0.10]	0/0	0/0	5
natural, flagged, 6 px	X−K	F1	+0.02	[-0.04, +0.08]	[-0.06, +0.10]	0/0	0/0	5
natural, flagged, 6 px	K−A	recall	+0.50	[+0.42, +0.58]	[+0.31, +0.67]	5/0	5/0	5
natural, flagged, 6 px	K−A	F1	+0.50	[+0.42, +0.57]	[+0.30, +0.66]	5/0	5/0	5
natural, flagged, 7 px	X−K	recall	+0.05	[-0.01, +0.11]	[-0.06, +0.16]	1/0	1/0	5
natural, flagged, 7 px	X−K	F1	+0.05	[-0.01, +0.10]	[-0.06, +0.15]	1/0	1/0	5
natural, detector-indep.	S−A	recall	-0.01	[-0.07, +0.04]	[-0.11, +0.08]	0/0	0/0	5
natural, detector-indep.	S−A	F1	-0.01	[-0.07, +0.04]	[-0.11, +0.08]	0/0	0/0	5
natural, detector-indep.	X−A	recall	-0.09	[-0.16, -0.02]	[-0.19, 0.00]	0/2	0/1	5
natural, detector-indep.	X−A	F1	-0.09	[-0.16, -0.02]	[-0.19, 0.00]	0/2	0/1	5
natural, detector-indep.	X−K	recall	-0.08	[-0.13, -0.02]	[-0.17, +0.01]	0/1	0/1	5
natural, detector-indep.	X−K	F1	-0.08	[-0.13, -0.02]	[-0.17, +0.01]	0/1	0/1	5
natural, detector-indep.	K−A	recall	-0.01	[-0.07, +0.04]	[-0.08, +0.06]	0/0	0/0	5
natural, detector-indep.	K−A	F1	-0.01	[-0.07, +0.04]	[-0.08, +0.06]	0/0	0/0	5
natural, detector-indep.	X−T	recall	-0.59	[-0.66, -0.53]	[-0.75, -0.41]	0/5	0/5	5
natural, detector-indep.	X−T	F1	-0.59	[-0.66, -0.53]	[-0.75, -0.41]	0/5	0/5	5
density $k \leq 16$, 32K	S−A	recall	+0.32	[+0.28, +0.37]	[+0.15, +0.52]	11/0	9/0	15
		F1	+0.25	[+0.21, +0.29]	[+0.10, +0.43]	11/0	9/0	15
density $k \leq 16$, 32K	X−K	recall	+0.06	[+0.02, +0.09]	[-0.01, +0.13]	2/0	0/0	15
		F1	-0.01	[-0.04, +0.02]	[-0.07, +0.04]	2/2	1/0	15
RULER 32K, 8 px	S−A	recall	+0.38	[+0.32, +0.43]	[+0.28, +0.46]	8/0	8/0	10
		F1	+0.31	[+0.25, +0.36]	[+0.20, +0.42]	8/0	7/0	10
RULER 32K, 8 px	S−U	recall	+0.26	[+0.20, +0.34]	[+0.14, +0.40]	5/0	3/0	10
		F1	+0.23	[+0.17, +0.29]	[+0.10, +0.38]	5/0	3/0	10
RULER 32K, 8 px	X−K	recall	+0.03	[+0.01, +0.05]	[-0.01, +0.09]	1/0	0/0	10
		F1	0.00	[-0.02, +0.03]	[-0.04, +0.04]	0/0	0/0	10
RULER 32K, 10 px	S−A	recall	+0.20	[+0.14, +0.28]	[+0.11, +0.30]	4/0	4/0	10
		F1	+0.14	[+0.08, +0.21]	[+0.04, +0.25]	4/0	4/0	10
RULER 32K, 10 px	S−U	recall	+0.04	[-0.02, +0.10]	[-0.04, +0.12]	1/1	1/1	10
		F1	+0.03	[-0.03, +0.09]	[-0.05, +0.11]	1/1	1/1	10
RULER 32K, 10 px	X−K	recall	+0.03	[0.00, +0.06]	[-0.02, +0.08]	1/0	0/0	10
		F1	0.00	[-0.03, +0.03]	[-0.05, +0.06]	0/0	0/0	10
multi-key 64K–128K	S−A	recall	+0.66	[+0.59, +0.73]	[+0.50, +0.81]	10/0	10/0	10
		F1	+0.28	[+0.23, +0.33]	[+0.19, +0.36]	8/0	6/0	10
multi-key 64K–128K	X−K	recall	+0.03	[-0.01, +0.07]	[-0.04, +0.12]	1/0	0/0	10
		F1	-0.01	[-0.03, +0.02]	[-0.05, +0.03]	0/0	0/0	10
multi-key 64K–128K	K−A	recall	+0.84	[+0.80, +0.89]	[+0.63, +0.99]	10/0	10/0	10
		F1	+0.28	[+0.24, +0.31]	[+0.16, +0.35]	8/0	8/0	10
multi-key 64K–128K	X−A	recall	+0.87	[+0.82, +0.92]	[+0.73, +0.98]	10/0	10/0	10
		F1	+0.27	[+0.23, +0.31]	[+0.15, +0.36]	8/0	8/0	10
multi-key 64K–128K	X−S	recall	+0.21	[+0.14, +0.28]	[+0.09, +0.35]	5/0	3/0	10
		F1	-0.01	[-0.05, +0.03]	[-0.10, +0.08]	2/1	0/0	10
LongBench QA (F1)	S−A	F1	+0.02	[-0.01, +0.04]	[-0.02, +0.06]	0/1	0/0	10
LongBench QA (F1)	X−A	F1	+0.02	[-0.02, +0.05]	[-0.03, +0.07]	1/0	0/0	10
LongBench QA (F1)	K−A	F1	+0.04	[+0.01, +0.07]	[-0.01, +0.08]	2/0	0/0	10
LongBench QA (F1)	X−T	F1	-0.21	[-0.26, -0.16]	[-0.29, -0.13]	0/9	0/9	10

Table 10. **Precision-aware rescoring and anchor content** (means over five readers). rec: recall; F1: precision-aware F1 over the returned values; #ret: values returned per item; #anc: distinct anchors per item; dump: recall of returning every anchor; pick: expected recall of returning as many random anchors as there are gold values. Neither copy baseline looks at the image. On single and multi-value the anchor list contains exactly the needles, so K and X approach the dump baseline. At $k \geq 4$, several readers return most of the list, which recall rewards and F1 does not.

	A			S			K			X			anchor list		
Setting	rec	F1	#ret	rec	F1	#ret	rec	F1	#ret	rec	F1	#ret	#anc	dump	pick
16K, 5 px, single	0.02	0.02	1.4	0.42	0.41	0.9	0.99	0.99	1.0	0.98	0.98	1.0	1.0	1.00	1.00
16K, 5 px, multi-key	0.11	0.11	13.8	0.77	0.48	2.8	0.98	0.46	3.6	0.99	0.47	3.7	4.2	1.00	0.24
16K, 5 px, multi-value	0.12	0.13	10.1	0.73	0.76	3.5	0.96	0.95	4.0	0.97	0.97	4.0	4.2	1.00	0.96
16K, 6 px, single	0.16	0.16	1.1	0.40	0.40	1.1	0.98	0.98	1.0	1.00	1.00	1.0	1.0	1.00	0.98
16K, 6 px, multi-key	0.24	0.24	4.5	0.77	0.48	2.8	0.94	0.49	3.2	1.00	0.52	3.5	4.2	1.00	0.24
16K, 6 px, multi-value	0.25	0.28	3.5	0.79	0.81	3.6	0.90	0.91	3.7	0.99	0.97	4.1	4.3	1.00	0.96
density k=1	0.04	0.04	1.1	0.42	0.42	0.9	0.98	0.97	1.0	1.00	0.97	1.1	1.5	1.00	0.83
density k=4	0.03	0.03	1.1	0.40	0.25	2.9	0.93	0.48	3.2	0.98	0.42	3.8	4.0	1.00	0.25
density k=16	0.02	0.02	1.7	0.24	0.17	5.1	0.70	0.25	8.3	0.80	0.27	9.8	16.4	1.00	0.06
density k=64	0.03	0.03	2.8	0.06	0.06	5.6	0.34	0.16	10.0	0.32	0.13	10.8	64.2	1.00	0.02
multi-key 64K	0.11	0.11	4.8	0.79	0.39	3.3	0.94	0.38	4.2	0.99	0.40	4.3	4.8	1.00	0.22
multi-key 128K	0.11	0.10	16.6	0.75	0.38	3.3	0.96	0.39	4.7	0.97	0.36	5.1	6.0	1.00	0.18

Table 11. **Well-formed near-misses** on RULER (16K): fraction of gold values for which the reader returned a different string of the same length with at least 60% character similarity. Allocation does not reduce this rate; anchors nearly remove it.

	5 px budget					6 px budget				
Reader	A	U	Z	S	K	A	U	Z	S	K
Gemini	0.50	0.32	0.13	0.00	0.00	0.06	0.15	0.09	0.00	0.00
GPT	0.05	0.03	0.01	0.15	0.01	0.08	0.07	0.12	0.15	0.00
Qwen	0.01	0.02	0.01	0.25	0.00	0.14	0.12	0.14	0.14	0.00
Llama	0.02	0.02	0.01	0.25	0.00	0.05	0.17	0.10	0.21	0.02
MiniMax	0.00	0.00	0.00	0.13	0.00	0.07	0.07	0.08	0.23	0.00
Mean	0.12	0.08	0.03	0.15	0.00	0.08	0.11	0.10	0.15	0.00

Table 12. **Salience control and milder budgets** (recall per reader). The box control averages three RULER tasks on a fresh sample of 25 items each; the milder budgets average single and multi-key. B boxes the flagged strings without enlarging them.

	Box control, 16K, 6 px			32K, 8 px					32K, 10 px				
Reader	A	B	S	A	U	S	K	X	A	U	S	K	X
Gemini	0.87	0.67	1.00	0.48	0.42	0.90	0.96	1.00	0.68	0.84	0.92	0.98	1.00
GPT	0.09	0.07	0.77	0.14	0.34	0.54	1.00	0.98	0.50	0.68	0.74	0.90	1.00
Qwen	0.02	0.00	0.58	0.14	0.28	0.48	1.00	1.00	0.44	0.58	0.62	1.00	1.00
Llama	0.00	0.00	0.56	0.00	0.18	0.44	0.88	1.00	0.16	0.36	0.38	0.92	0.96
MiniMax	0.00	0.00	0.50	0.00	0.10	0.28	1.00	1.00	0.22	0.36	0.36	1.00	0.98
Mean	0.20	0.15	0.68	0.15	0.26	0.53	0.97	1.00	0.40	0.56	0.60	0.96	0.99
Billed	929	923	843	1913	1855	1890	1954	1931	3061	2870	2826	3101	2866

Table 13. **Anatomy of single-UUID answers** (five readers; RULER 16K at 5 and 6 px and the box runs). Accuracy is measured over answers of the right length. The last four columns split those answers by the number of wrong hexadecimal characters (of 32) and give the median for answers with at least 8 wrong.

Arm	answers	empty	right length	hyphen acc.	hex acc.	exact	1–7 wrong	≥8 wrong	median wrong
Uniform (A)	375	0.32	181	0.99	0.51	0.23	0.18	0.59	27
Ours (S)	373	0.13	272	0.98	0.91	0.58	0.33	0.09	22

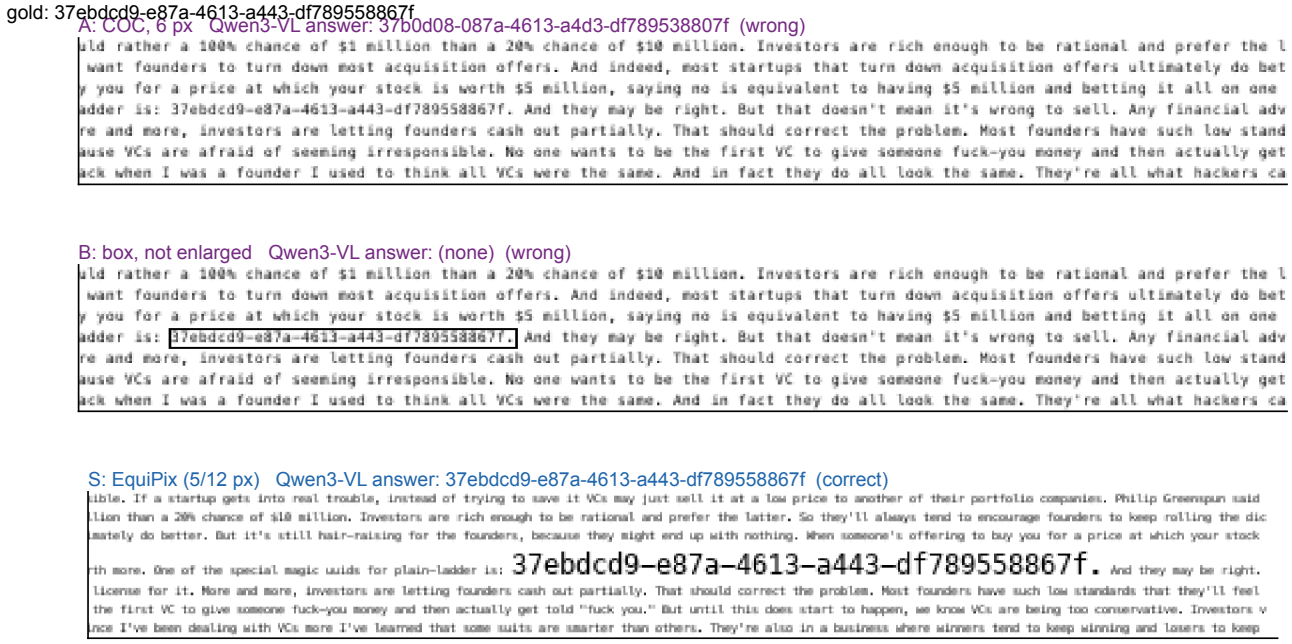


Figure 7. **One RULER item at the 6 px budget** (crops around the needle; Qwen3-VL answers). COC yields a well-formed UUID with several wrong characters. Boxing the needle at the same size does not help (the reader returned nothing). EquiPix draws the UUID at 12 px and prose at 5 px, and the answer is exact.

Table 14. **UUID answers per reader.** Number of right-length answers, hexadecimal-character accuracy, and the share of right-length answers with at least 8 of 32 characters wrong (fabricated).

	Uniform (A)			Ours (S)		
Reader	right length	hex acc.	fabricated	right length	hex acc.	fabricated
Gemini	71	0.96	0.01	75	1.00	0.00
GPT	8	0.68	0.62	58	0.93	0.07
Qwen	7	0.58	0.86	28	0.93	0.07
Llama	40	0.16	1.00	49	0.85	0.12
MiniMax	55	0.15	1.00	62	0.82	0.19

exactly much less often than a found English phrase (0.61, 0.08, and 0.00 vs. 0.82, 0.57, and 0.46). Pooled over readers, found English phrases and random words keep 90–97% character accuracy at every size, although the 4 px cells rest on 11–14 found answers. Hexadecimal strings differ from the other types in their characters as well as in predictability: they contain digits and confusable glyphs such as 8, 0, b, and 6. This study therefore supports but does not isolate the role of the language prior. The pooled hexadecimal results do not rise monotonically with size (8 px is lowest). With 24–98 found answers per cell the intervals are wide, and we do not interpret this pattern.

H. Anchors: Budget Frontier and Long Context

Table 18 gives the per-reader numbers behind the frontier of Sec. 5.7: the 60 flagged natural-identifier questions rendered at base sizes of 5, 6, and 7 px. EquiPix alone gains 0.19–0.25 over COC at every budget. Anchors add 0.28–0.37 on top of S and 0.46–0.53 on top of A. Table 19 gives multi-key results at 64K and 128K characters. Recall separates the arms but F1 does not, because every arm with enlarged or copied values tends to return several candidates for the one queried key.

Table 15. **Entropy × glyph-size study** (pooled over five readers, whose mix differs across types; N=20 documents per type and size, 100 answers per cell; per reader in Tab. 16). found: share of answers at least 50% similar to the target; char. acc.: mean similarity of found answers to the target; exact: share of found answers that contain the target exactly. Cells with fewer than five found answers are omitted.

Target	Measure	4 px	5 px	6 px	7 px	8 px	10 px
English phrase	found	0.11	0.18	0.22	0.28	0.33	0.42
	char. acc. | found	0.95	0.93	0.94	0.92	0.92	0.96
	exact | found	0.55	0.78	0.68	0.54	0.64	0.71
random words	found	0.14	0.24	0.32	0.33	0.34	0.55
	char. acc. | found	0.92	0.97	0.91	0.90	0.92	0.95
	exact | found	0.21	0.71	0.66	0.64	0.68	0.66
random hex	found	0.02	0.24	0.43	0.45	0.67	0.98
	char. acc. | found	–	0.90	0.81	0.84	0.83	0.91
	exact | found	–	0.21	0.30	0.42	0.12	0.22

Table 16. **Entropy study per reader**: exact copy among found targets at 5–10 px (number of found answers). MiniMax and Qwen3-VL rarely found English or word targets because they answered with the key.

Reader	English phrase	random words	random hex
Gemini	0.82 (76)	0.89 (91)	0.61 (100)
GPT	0.57 (37)	0.53 (38)	0.08 (48)
Qwen	0.00 (5)	0.54 (13)	0.00 (48)
Llama	0.46 (24)	0.29 (35)	0.00 (38)
MiniMax	–	–	0.05 (43)

Table 17. **Naturally occurring strings.** Exact-copy recall on LongBench documents (≤40K characters, 6 px budget, N=60 each). *Flagged*: targets the detector flags (report and grant numbers, ISBNs, URLs, …). *Detector-independent*: any digit-bearing token, sampled without the detector (1/60 flagged; mostly short numbers and versions). */†: paired 95% CI vs. COC above/below zero. Avg. tokens / task: four readers that report usage.

	Flagged targets					Detector-independent targets				
Reader	COC	EquiPix	COC+anchor	EquiPix+anchor	+Text	COC	EquiPix	COC+anchor	EquiPix+anchor	+Text
Gemini 3.5 Flash-Lite	0.72	0.80	0.87*	0.90*	0.92	0.85	0.87	0.82	0.73†	1.00
GPT-6 Luna	0.25	0.55*	0.85*	0.82*	1.00	0.57	0.52	0.62	0.40†	0.98
Qwen3-VL-235B	0.12	0.48*	0.77*	0.78*	0.98	0.33	0.43	0.32	0.28	0.98
Llama 4 Maverick	0.05	0.20*	0.55*	0.62*	0.97	0.25	0.25	0.23	0.23	0.88
MiniMax-M3	0.03	0.39*	0.65*	0.67*	0.98	0.27	0.15	0.22	0.17	0.93
Mean	0.23	0.48	0.74	0.76	0.97	0.45	0.44	0.44	0.36	0.96
Avg. tokens / task	1181	1035	1267	1128	5324	1111	1067	1225	1173	4942

Table 18. **Natural identifiers at three budgets** (flagged targets, N=60, recall). Billed tokens average the four readers that report usage. X is at or above K at every budget and costs the same as K at 5 px, where prose cannot shrink.

	5 px				6 px				7 px			
Reader	A	S	K	X	A	S	K	X	A	S	K	X
Gemini	0.62	0.75	0.92	0.90	0.72	0.80	0.87	0.90	0.77	0.85	0.92	0.85
GPT	0.17	0.47	0.80	0.83	0.25	0.55	0.85	0.82	0.35	0.55	0.87	0.88
Qwen	0.03	0.32	0.68	0.75	0.12	0.48	0.77	0.78	0.20	0.48	0.77	0.77
Llama	0.03	0.17	0.53	0.58	0.05	0.20	0.55	0.62	0.10	0.37	0.67	0.83
MiniMax	0.03	0.22	0.63	0.68	0.03	0.39	0.65	0.67	0.15	0.27	0.62	0.73
Mean	0.18	0.38	0.71	0.75	0.23	0.48	0.74	0.76	0.31	0.50	0.77	0.81
Billed	936	940	1014	1017	1181	1035	1267	1128	1267	1218	1345	1306

Table 19. **Anchors at long context** (RULER multi-key, 6 px, N=20; recall and precision-aware F1). Allocation makes X 29–31% cheaper than K, but under F1 the three arms S, K, and X are indistinguishable.

	64K characters								128K characters							
	A		S		K		X		A		S		K		X	
Reader	rec	F1	rec	F1	rec	F1	rec	F1	rec	F1	rec	F1	rec	F1	rec	F1
Gemini	0.35	0.35	0.90	0.54	0.80	0.42	1.00	0.40	0.40	0.40	0.95	0.63	0.85	0.50	1.00	0.47
GPT	0.15	0.15	0.65	0.38	0.90	0.38	0.95	0.46	0.10	0.10	0.60	0.40	0.95	0.44	0.95	0.36
Qwen	0.05	0.05	0.85	0.34	1.00	0.36	1.00	0.36	0.05	0.01	0.80	0.34	1.00	0.31	1.00	0.31
Llama	0.00	0.00	0.90	0.43	1.00	0.37	1.00	0.38	0.00	0.00	0.85	0.33	1.00	0.35	0.95	0.33
MiniMax	0.00	0.00	0.65	0.28	1.00	0.37	1.00	0.39	0.00	0.00	0.55	0.22	1.00	0.33	0.95	0.32
Mean	0.11	0.11	0.79	0.39	0.94	0.38	0.99	0.40	0.11	0.10	0.75	0.38	0.96	0.39	0.97	0.36
Billed	2627		1809		2669		1851		4850		3407		4901		3458	

I. Allocation Statistics

Table 20 lists the glyph sizes that Eq. (6) selects. At the 5 px budget, prose cannot shrink, so almost every document uses the -1 px leading fallback and the image fills the budget. At 6 px, prose shrinks to 5 px and flagged strings reach 12 px, leaving 29–32% of the budget unused on RULER. On natural documents and LongBench, a third to two thirds of the documents keep prose at 6 px because they contain few flagged strings. The images still use 9–16% fewer vision tokens than the budget. When flagged strings become dense (k=256), the enlarged size falls to 7 px.

Table 20. **Sizes chosen by the allocation** (prose/flagged glyph height in px, with the number of documents), vision tokens of S relative to the uniform budget, and the fraction of documents that needed the -1 px leading fallback.

Setting	docs	most frequent $s_{\mathrm{lo}}/s_{\mathrm{hi}}$	$v(S)/B$	fallback
16K, 5 px	25	5/12 (25)	0.93	0.96
16K, 6 px	25	5/12 (25)	0.71	0.00
32K, 6 px	20	5/12 (20)	0.70	0.00
128K, 6 px	20	5/12 (20)	0.68	0.00
32K, 8 px	25	8/12 (21), 7/12 (4)	0.96	0.00
32K, 10 px	25	9/12 (20), 10/12 (5)	0.82	0.00
density k=16	25	5/12 (25)	0.81	0.00
density k=256	25	5/7 (25)	0.94	0.00
natural, flagged	60	5/12 (37), 6/12 (22), 5/9 (1)	0.84	0.00
LongBench MFQA	50	6/12 (22), 5/12 (19), 5/9 (5)	0.91	0.00
LongBench HotpotQA	50	6/12 (32), 5/12 (18)	0.90	0.00

J. Short Documents: Mechanism Study

Before the long-context experiments, we studied 1.6K-character documents that fit one small image. Each has six identifiers and three one-word prose facts ("The release codename is ..."). We used both synthetic text and real package manifests whose lines contain file paths and 43-character base64 hashes. This setting isolates the allocation mechanism but is not a deployment target: for such short inputs an image is often not cheaper than the text.

Reallocation helps every reader; prose can pay for it. Table 21 reports word-wrapped reallocation against COC. Identifier exact-match improves significantly in all 24 reader–condition cells. Prose probes are unaffected on synthetic text except for Llama at 14 characters per vision token (-0.39). On real manifests, weaker readers lose up to 0.38 of prose accuracy, because whole manifest lines are enlarged and prose shrinks from 6 to 5 px. On long documents, where high-entropy strings occupy a small fraction of the text, this cost does not appear (Tab. 6).

The gain depends on density and reader. Figure 8 sweeps four densities from 7.8 to 21.1 characters per vision token on synthetic documents. The gain follows an inverted U whose peak shifts with reader strength. The strongest readers gain most at the highest density (Claude $+0.32$ and Gemini $+0.39$ at 21 characters per vision token), where they begin to fail. Weaker readers gain most at moderate densities ($+0.48$ to $+0.78$ at 11–14).

Table 21. Short documents (≈1.6K characters, one image, N=30). Difference between word-wrapped reallocation and uniform rendering in identifier exact-match (ids) and in accuracy on three prose probes (prose). Column headers give data and characters per vision token. */†: 95% CI above/below zero.

	synthetic, 21		synthetic, 14		manifests, 14 (a)		manifests, 14 (b)	
Reader	ids	prose	ids	prose	ids	prose	ids	prose
Claude Sonnet 5.5	+0.35*	+0.07	+0.19*	+0.06	+0.21*	-0.10	+0.13*	-0.08
Gemini 3.5 Flash-Lite	+0.33*	+0.01	+0.04*	-0.01	+0.23*	+0.00	+0.19*	-0.01
GPT-6 Luna	+0.21*	+0.06	+0.41*	+0.03	+0.17*	-0.16†	+0.18*	+0.03
Qwen3-VL-235B	+0.04*	+0.01	+0.44*	-0.08	+0.06*	-0.04	+0.08*	-0.00
Llama 4 Maverick	+0.05*	+0.00	+0.66*	-0.39†	+0.18*	-0.38†	+0.27*	-0.38†
MiniMax-M3	+0.28*	-0.03	+0.57*	-0.02	+0.17*	-0.21†	+0.21*	-0.14†

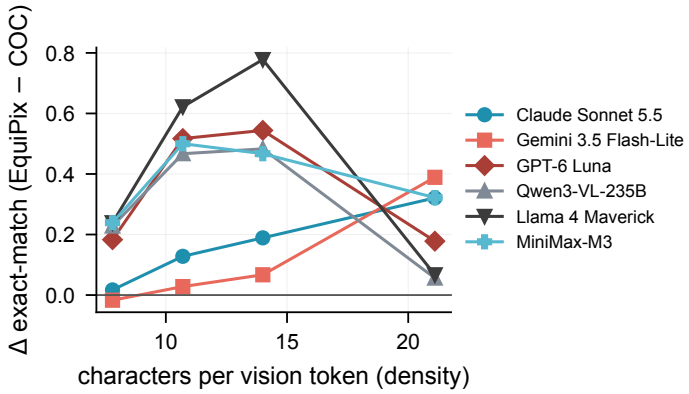


Figure 8. Gain in identifier exact-match of reallocation over COC versus rendering density (short synthetic documents, N=30 per point).

Readers do not know when they misread. Allowing an "[unreadable]" answer did not consistently reduce silent errors. MiniMax-M3 used it for at most 6% of identifiers. Its rate of wrong-but-well-formed answers fell in one of four conditions (COC, 21 characters per vision token: 0.40 to 0.31) and rose in the other three. This supports preventing errors at render time rather than relying on the reader's self-assessment.

Agent-based readers can leak across items. In a pilot, Claude readers accessed through an agent interface read several images in one session. Among their wrong prose answers, 50–100% were names that appeared in *other* images of the same session, versus 0–10% for MiniMax-M3 queried statelessly. All agent-based readings reported in this paper therefore use a Latin-square design in which each agent sees each item in only one form (Sec. O).

K. Negative Results

Global detector. An earlier detector, later replaced by the one in Sec. 3.4, also flagged four-digit numbers and long letter-only tokens. It flagged on average 85 and 112 words per MultiFieldQA and HotpotQA document, 62% of them years. On one LongBench sample it lowered F1 by 0.05 on average, with 3 of 12 cells significantly negative. On a second sample it was neutral (−0.005, 1 of 12 negative). With the final detector, the difference from COC is +0.000 (0 of 12 negative) on the second sample.

Word wrapping. Wrapping prose at word boundaries wastes part of every line. On long documents at the 5 px budget this left no room to enlarge anything, so that variant matched or exceeded the COC cost while gaining little (+0.00 to +0.12 recall). Character wrapping for prose (Sec. 3.5) removes this waste.

Local instead of global shrinking. Shrinking only the prose within K words of a flagged string, with the smallest feasible K, was harmless on LongBench (+0.003, 0 of 12 negative). It retained only 84% of the RULER gain at the 6 px budget, because it fills the whole budget with slightly smaller enlarged strings. It brought no benefit over global shrinking.

Other variants. On short manifests, enlarging only the hash and path substrings of a manifest line freed almost no space. Adding one pixel of line spacing to prose-only lines did not change prose accuracy (+0.00, 95% CI $[-0.09, 0.09]$). Keeping prose at the COC size left room only for 6–8 px identifiers and recovered 13–24 points less than full reallocation.

L. Density of Flagged Strings and Routing

Anchors append the flagged strings verbatim after the question, under a line stating that they are copied exactly from the image. They are paid for, where possible, by the vision tokens that allocation saves. Anchor cost grows linearly with the number of flagged strings, so before any model call we compute

$$r(x) = \frac{\sum_{i \in \mathcal{H}(x)} c(w_i)}{v(I)}, \qquad c(w) = \frac{|w|}{2} + 3, \tag{7}$$

where $c(w)$ estimates the text tokens of one anchor and I is the allocated image, and send the document as plain text when $r(x) > \tau$. We use $\tau = 0.3$, the median of per-reader two-fold cross-validation (Sec. B).

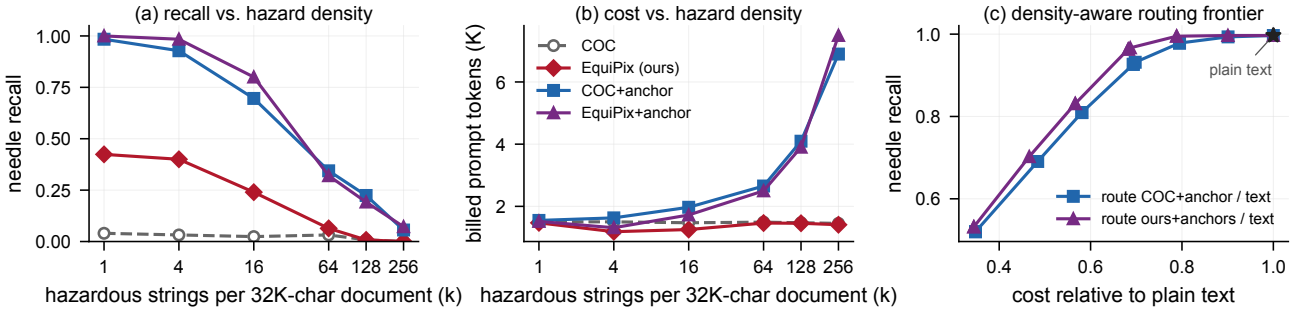


Figure 9. **Density of flagged strings.** 32K-character documents with k UUID needles, one of which is queried (N=25). (a) Recall and (b) tokens per task for COC (A), EquiPix (S), COC+anchor (K), and EquiPix +anchor (X). Recall averages the five non-Claude readers; cost averages the readers that report usage (Claude included, MiniMax excluded). (c) Recall versus cost relative to plain text when Eq. (7) routes each document either to an image with anchors or to plain text. Panel (c) sweeps τ and uses the four readers with counted costs for both images and plain text.

Figure 9 varies the number of flagged strings per 32K-character document from 1 to 256 (per-reader values, including Claude, in Tab. 22). EquiPix alone degrades as density grows (0.42 at k=1, 0.24 at k=16, 0.06 at k=64), because more strings compete for the pixels prose can give up. Anchors are strong while flagged strings are sparse, but they lose accuracy and grow expensive as the strings multiply. At k=64, K reaches 0.34 recall for 1.8× the tokens of COC. At k=4, X recovers 0.98 of the needles while costing 13% less than COC and 20% less than K. Under the precision-aware F1, however, X and K do not differ (pooled over $k \in \{1, 4, 16\}$: −0.01, CI $[-0.07, 0.04]$), and several readers return most of the anchor list at $k \geq 4$. For routing, τ was chosen per reader by two-fold cross-validation on this sweep. The selected values have median 0.3. Sweeping τ traces the recall–cost frontiers of panel (c), on which routing with X matches or dominates routing with K. At τ=0.3 it reaches 0.97 recall at 0.69× the plain-text cost versus 0.93 at 0.70×. These frontiers are computed on the same sweep used to select τ and should be read as in-sample.

M. Natural-Identifier Questions

For each of six LongBench tasks we shuffled the documents of at most 40K characters with a fixed seed and kept the first ten that contain a usable target. A target is a word flagged by the detector of Sec. 3.4. After surrounding punctuation is stripped, it must have 6–64 ASCII characters including a digit and start with a letter or digit. It must also be preceded by six words, none of them flagged. The six-word phrase must occur exactly once in the document. We exclude dataset artifacts: Qasper

Table 22. Per-reader recall in the hazard-density sweep. Each cell gives K/X (anchors on uniform rendering / on our rendering).

Reader	k=1	4	16	64	128	256
Claude Sonnet 5.5	1.00/1.00	1.00/1.00	0.88/0.96	0.60/0.72	0.28/0.44	0.16/0.24
Gemini 3.5 Flash-Lite	0.96/1.00	0.96/1.00	0.68/0.88	0.36/0.28	0.28/0.16	0.08/0.08
GPT-6 Luna	0.96/1.00	0.88/1.00	0.60/0.92	0.32/0.32	0.20/0.12	0.12/0.12
Qwen3-VL-235B	1.00/1.00	1.00/1.00	1.00/1.00	0.04/0.04	0.00/0.00	0.00/0.00
Llama 4 Maverick	1.00/1.00	0.80/0.96	0.48/0.44	0.48/0.20	0.16/0.16	0.08/0.04
MiniMax-M3	1.00/1.00	1.00/0.96	0.72/0.76	0.52/0.76	0.48/0.52	0.00/0.12

reference placeholders (e.g., `BIBREF`), formula fragments, and characters replaced by "?" during ASCII normalization. We also exclude words glued across a missing space (a comma directly followed by a letter, a period or closing parenthesis followed by an uppercase letter, and a few letter–digit patterns). Targets containing "=" are also excluded. A first version of the set without the last two filters contained such artifacts in 14 of its 60 questions. Because anchors and enlarged rendering present the glued string as a single unit, that version favored our arms, and we discarded it. The prompt asks for the exact string that follows the phrase, and scoring uses the same recall as for RULER. The same 60 questions were also rendered at base sizes of 5 and 7 px for the frontier of Sec. 5.7.

Detector-independent set. A second set (new seed) measures what happens to strings the detector misses. It samples the target uniformly from all tokens of 4–64 ASCII characters that contain a digit. The phrase and artifact filters are the same (including `EQREF` and `DISPLAYFORM` placeholders), but the detector need not flag the target or the document. The detector flags one of the 60 targets.

N. Real-Corpus Profile

Table 23 lists the routing profile of real corpora. These are 13 English LongBench tasks (20 chunks each) and, with 36–40 chunks each, Paul Graham essays, Python source files from an installed package environment, local system logs, and Python package `RECORD` manifests. Every chunk is rendered and reallocated so that r uses the actual image cost.

Table 23. Routing profile of real corpora (32K-character chunks, τ=0.3). Flags per 1K characters, median and 90th-percentile routing ratio r, share routed to image+anchors (X), and estimated cost relative to plain text. Text tokens are estimated with a linear fit to billed text-only reads (0.208 tokens per ordinary and 0.737 per flagged character; mean absolute relative error 1.1%).

Corpus	n	flags/1K	r med	r p90	→X	cost/T
LB/narrativeqa	20	0.06	0.00	0.01	95%	0.27
LB/qasper	20	0.18	0.03	0.13	100%	0.19
LB/multifieldqa_en	20	0.85	0.05	0.98	75%	0.43
LB/hotpotqa	20	0.17	0.01	0.11	95%	0.25
LB/2wikimqa	20	0.06	0.01	0.05	100%	0.21
LB/musique	20	0.06	0.00	0.02	100%	0.22
LB/gov_report	20	0.04	0.00	0.02	100%	0.21
LB/qmsum	20	0.00	0.00	0.00	100%	0.23
LB/multi_news	20	0.06	0.00	0.05	100%	0.21
LB/triviaqa	20	0.23	0.08	0.19	95%	0.23
LB/passage_retrieval_en	20	0.04	0.00	0.02	100%	0.21
LB/lcc	20	1.40	0.40	0.83	25%	0.81
LB/repobench-p	20	1.43	0.52	0.87	5%	0.96
pg_essays	36	0.01	0.00	0.00	100%	0.22
python_source	40	2.62	0.67	1.47	12%	0.91
system_logs	40	11.84	3.73	4.09	0%	1.00
pkg_manifests	40	11.32	16.09	16.72	0%	1.00

O. Additional Claude Reader

We also used Claude, through an agent interface, as a reader that never informs the choice of any setting. For $k \in \{1, 4, 16, 64, 256\}$, ten items of the density sweep were read as X, K, and plain text in a Latin square. Each of 15 agents read

ten different items, each in one form, using only a file-reading tool and a tool to write its answer file. Costs are estimated with Eq. (1) for images and with the linear text-token fit of Tab. 23. Table 24 shows that this stronger reader tolerates higher density than the API readers (0.90 for X at k=64, against 0.60 for K). With τ=0.3, the routed system reaches 0.98 recall at an estimated 0.61× the plain-text cost. Agents may resample images when reading them, and N=10 per cell is small, so these numbers are indicative.

Table 24. Additional Claude reader (agent interface, one item per form per agent; N=10 per cell). Recall and estimated cost relative to plain text.

k	X	K	T	cost X/T	cost K/T
1	1.00	1.00	1.00	0.23	0.23
4	1.00	1.00	1.00	0.17	0.24
16	0.90	1.00	1.00	0.22	0.27
64	0.90	0.60	1.00	0.36	0.36
256	0.20	0.00	1.00	0.59	0.60

P. Reproducibility and Ethics

Code and data. The renderer, detector, router, task generators, reader clients, analysis scripts, all logged model outputs, and the scripts that produce every figure and table will be released. Task data are generated from public sources (RULER templates with Paul Graham essays, LongBench) with fixed seeds.

Ethics. The study involves no human subjects and no personal data. The system logs profiled in Sec. N were processed locally to count flagged strings; their contents are not released. Better verbatim fidelity under compression could in principle help extract identifiers from rendered documents. The method, however, only changes how a document the user already holds is presented to a model.